

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DATABASE SYSTEM : ASSIGNMENT

Digital Distribution Platform: St

Final Report: Database Design, SQL Implementation, Stored Programs, T

Group: 6

Instructor: Dương Huỳnh Anh Đức

Students:	Phan Huy Long	2352697	CC03
	Phạm Thành Nam	2352786	CC03
	Trần Phan Đăng Khôi	2352626	CC03
	Triệu Thiên Ân	2452113	CC03
	Nguyễn Gia Hùng	2352422	CC03

HO CHI MINH CITY, MAY 2026



Member list & Workload

No.	Name	Student ID	Main responsibilities	Contribution
1	Phan Huy Long	2352697	Developed Sections 1.1, 4.1 (table implementation, integrity constraints), and 5.2 (trigger computation for derived attributes). Also reviewed the consistency between the application features and the database design from previous parts.	100%
2	Phạm Thành Nam	2352786	Developed Section 5.4 (stored functions for business calculations) and finalized the overall report. Compiled and finalized the Part III report including organizing screenshots, descriptions, and captions.	100%
3	Triệu Thiên Ân	2452113	Developed Sections 5.1 (insert/update/delete stored procedures) and 6.1 (CRUD interface of the application).	100%
4	Nguyễn Gia Hùng	2352422	Developed Sections 4.2 (sample data and derived attribute analysis) and 6.2 (data list interface).	100%
5	Trần Phan Đăng Khôi	2352626	Developed Sections 1.2, 1.3, 2, 3, 5.3 (data retrieval procedures), and 6.3 (procedure/function demonstration interface).	100%

Table 1: Member list and workload for the merged Part II and Part III report



Contents

1	Data Requirements Analysis and Description	5
1.1	Research Related Applications/Systems for Reference	5
1.1.1	Rationale for choosing Steam as the primary reference system	5
1.1.2	Scope of Observation and Survey Approach	5
1.1.3	Business Analysis of Steam in Relation to the Proposed System	6
1.2	Proposed system	15
1.2.1	Description	15
1.2.2	Proposed Application Overview & Users	15
1.3	Semantic Constraints Not Fully Represented in the EERD	18
1.4	Full System Specification	19
2	EERD design	22
3	Mapping	23
4	Database Implementation	26
4.1	Implementing Tables and Integrity Constraints	26
4.1.1	Full SQL Implementation	26
4.1.2	Execution Result	31
4.2	Sample Data Insertion	32
4.2.1	Sample Data Coverage	32
4.2.2	Data Design Explanation	33
4.2.3	Full SQL Data Insertion Script	34
4.2.4	Execution Result	41
5	Stored Procedures, Triggers, and Functions	42
5.1	Stored Procedures for Insert, Update, and Delete	42
5.1.1	Selected Table and Operations	42
5.1.2	Procedure-level Validation	42
5.1.3	Full SQL Implementation	43
5.1.4	Test Cases	48
5.1.5	Execution Result	49
5.2	Triggers	51
5.2.1	Business Constraint Trigger	51
5.2.2	Chosen Derived Attribute	56
5.2.3	DML Operations Affecting the Derived Attributes	56
5.2.4	Full SQL Implementation	57



5.2.5	Trigger Explanation	60
5.2.6	Test Cases	61
5.2.7	Execution Result	63
5.3	Stored Procedures for Data Retrieval	64
5.3.1	Procedure 1: Get Games by Genre and Price	64
5.3.2	Procedure 2: Get Developer Revenue	65
5.3.3	Test Cases	66
5.3.4	Execution Result	66
5.4	Stored Functions	67
5.4.1	Function 1: Calculate Player Spending	67
5.4.2	Function 2: Calculate Developer Net Revenue	68
5.4.3	Full SQL Implementation	68
5.4.4	Test Cases	73
5.4.5	Execution Result	73
6	Application Implementation	75
6.1	CRUD Interface	75
6.1.1	Interface Design	75
6.1.2	Backend Connection	76
6.2	Data List Interface	77
6.2.1	Interface Overview	77
6.2.2	Search and Filtering	78
6.2.3	Sorting	79
6.2.4	Update and Delete Directly from the List	79
6.2.5	Input Validation and Error Handling	79
6.2.6	Execution Result	80
6.3	Procedure and Function Demonstration Interface	83
6.3.1	Interface Purpose	84
6.3.2	Frontend Design	84
6.3.3	Backend Processing	84
6.3.4	Stored Procedure Used	85
6.3.5	Execution Result	85

1 Data Requirements Analysis and Description

1.1 Research Related Applications/Systems for Reference

1.1.1 Rationale for choosing Steam as the primary reference system

Steam was selected as the primary reference system because it closely matches the scope of the proposed application. Rather than functioning only as an online game store, Steam operates as an integrated digital distribution platform that combines user accounts, storefront browsing, purchasing, personal library management, and community interaction. This makes it highly relevant as a reference for a system that must support both commercial transactions and long-term user engagement.

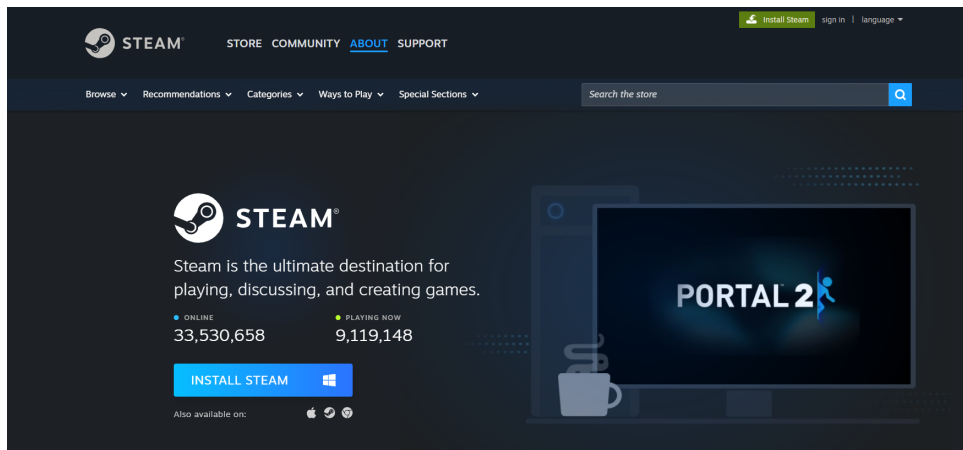


Figure 1.1: Steam platform overview page

The interface illustrates that Steam functions as an integrated digital distribution and community platform rather than a standalone online storefront. This makes it a suitable primary reference system for the proposed system.

1.1.2 Scope of Observation and Survey Approach

The group surveyed Steam from a data-oriented and system-analysis perspective rather than treating it solely as an end-user application. The purpose of this survey was to identify the types of data, user actions, and business relationships reflected in the platform's interface structure, displayed content, and interaction flows.

The scope of observation focused on the main areas of Steam that are directly relevant to the proposed system, including user account interfaces, storefront navigation, game detail pages, cart and checkout processes, personal library management, achievement tracking, reviews, and community-related features. In addition, the group also observed evidence of developer/publisher involvement through product page information such as publisher details, downloadable content (DLC), pricing, and promotional structures. User Management and Account-

Based Access Each observed element was treated as a basis for inferring data storage requirements, entity types, attributes, relationships, and business rules. As a result, the proposed system is grounded in a real-world platform that can be clearly observed and systematically analyzed.

1.1.3 Business Analysis of Steam in Relation to the Proposed System

This section should analyze **how Steam operates as a real-world information system, what business processes can be observed from it, and why those processes provide a suitable foundation for the proposed system.** Instead of merely listing features, the section should explain how Steam supports content distribution, purchasing, ownership management, community interaction, and publisher-side participation.

User Management and Account-Based Access Steam operates as a digital platform built around user accounts, in which most core activities are tied to a persistent user identity. To access personalized services, users must sign in with their account credentials or use alternative authentication methods such as QR-based login. This shows that account management is a fundamental business function of the platform rather than merely a supporting feature.

From a business perspective, account-based access enables Steam to provide secure and personalized services. In particular, a signed-in user can:

- purchase digital products through the platform
- access and manage a personal game library
- maintain a personal profile
- submit reviews and other user-generated content
- track achievement progress
- interact with community and social features

The login interface also suggests the existence of several authentication-related data elements, including:

- account name
- password credentials
- session persistence
- account recovery support

In addition, Steam's account-centered structure implies that the platform must support multiple types of users with different interaction purposes. While players use their accounts to consume content and participate in the community, the broader structure of the platform also reflects the participation of developer/publisher-side users through product and publishing activities. Therefore, account management is significant not only for authentication, but also for enabling role-based interaction across the platform.

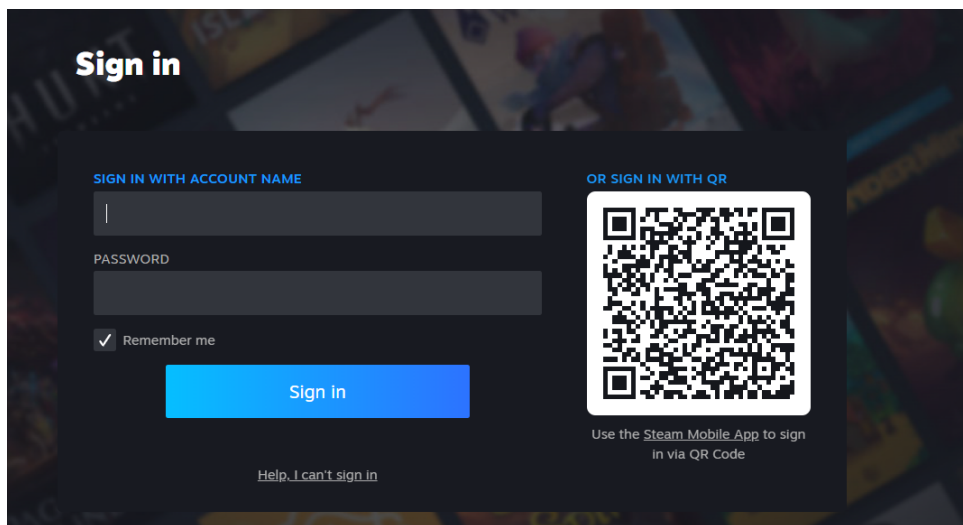


Figure 1.2: Steam sign-in interface

This interface indicates that Steam relies on account-based access to connect each user with personalized platform activities, including transactions, owned content, and social interaction.

Digital Product Presentation and Content Discovery Steam functions not only as a platform for selling digital games, but also as a structured environment for presenting and organizing game content in ways that support discovery and purchase decisions. Through its storefront, users can browse games by category, genre, tag, recommendation, popularity, or promotional event. This shows that the platform must manage a large amount of descriptive and classificatory information in order to help users navigate a broad digital catalog efficiently. From a business perspective, this function is essential because it connects available products with user interest and directly influences purchasing behavior.

Each game page on Steam presents multiple layers of information beyond the game title itself. These typically include:

- price
- release date
- developer

- publisher
- descriptive text
- screenshots and trailers
- tags and genres
- system requirements.

Such presentation indicates that a game on Steam is treated as a complex digital product rather than a simple item for sale. In addition, these data elements help users evaluate whether a game matches their preferences and technical conditions before making a purchase. Therefore, the storefront is not merely a visual interface, but a business mechanism for organizing product data and supporting content discovery.

From a system analysis perspective, this observed structure provides a clear basis for later data modeling in the proposed system. In particular, it suggests the need for:

- entities related to games,
- classification mechanisms such as genres and tags,
- developer/publisher associations, and
- product presentation attributes.

As a result, Steam serves as a strong reference for understanding how digital products should be structured, displayed, and connected to user-facing discovery functions in the proposed platform.

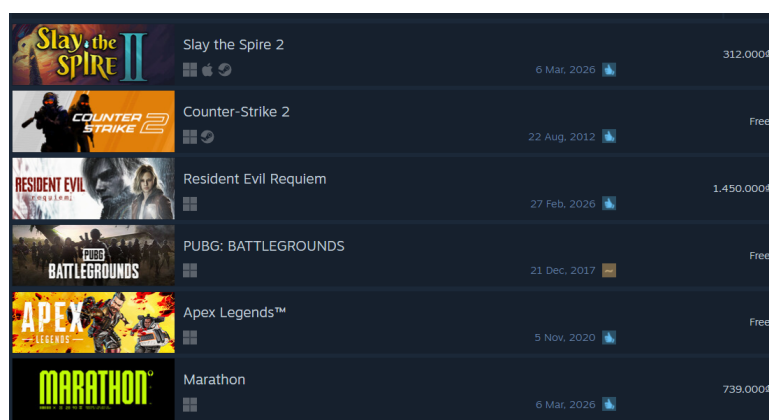


Figure 1.3: Steam storefront browsing interface

The Steam storefront organizes digital products through structured listing information such as titles, release dates, and prices, thereby supporting content discovery and user browsing behavior.

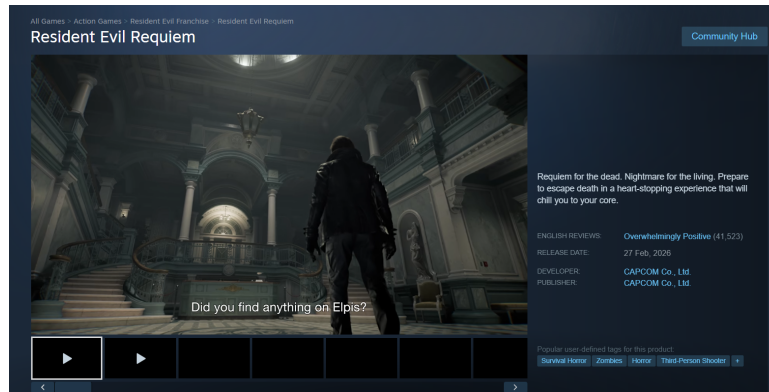


Figure 1.4: Steam game detail page

The Steam game detail interface illustrates that a digital product is represented through multiple layers of information, such as media presentation, review indicators, release information, developer/publisher details, and tags, thereby supporting structured content discovery and purchase consideration.

Cart, Checkout, and Transaction Processing Steam also demonstrates a complete digital purchasing workflow in which a player can select products, place them in a shopping cart, and proceed to checkout before a purchase is finalized. This shows that the platform does not treat purchasing as a single instant action, but as a multi-step business process.

From a system perspective, the purchasing workflow can be divided into two main stages:

- the cart, which represents temporary purchase intent
- the completed checkout process, which creates a formal transaction record

This distinction is important because it reflects two different stages of commercial activity: one for items that a user is considering buying and another for purchases that have been confirmed and recorded by the system.

In addition, Steam's purchasing flow shows that transaction-related data is influenced by contextual factors such as product pricing, discounts, and promotional events. The platform therefore manages several separate but related commercial components:

- cart contents
- checkout processing
- completed transaction records
- dynamic pricing conditions

The final purchase record must preserve not only the selected games, but also the price conditions applied at the time of purchase. As a result, Steam provides a strong reference for modeling the commercial workflow of the proposed system, especially for elements related to cart management, purchase records, payment processing, and promotion-aware transactions.

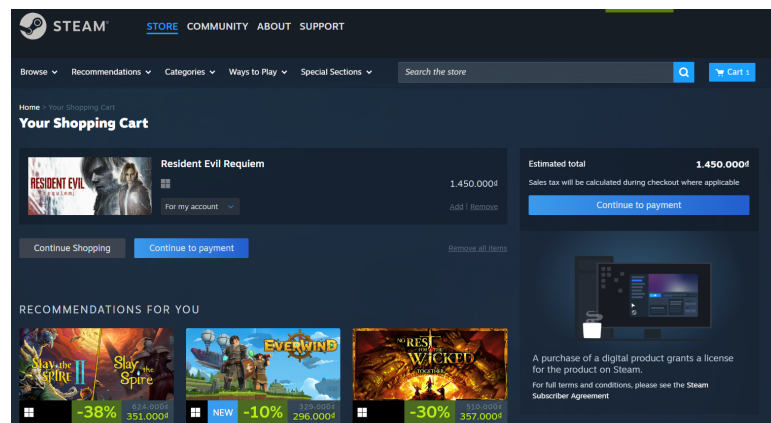


Figure 1.5: Steam shopping cart interface

This interface shows that Steam manages temporary purchase intent through a shopping cart before the checkout stage. It also presents pricing information, selected items, and a transition to payment, indicating that cart handling and transaction confirmation are treated as separate stages in the purchasing workflow.

Ownership Management and Personal Library After Purchase A key business characteristic of Steam is that the user experience does not end once a purchase is completed. After a transaction is finalized, the purchased game is added to the player's personal library, where it remains accessible for future installation, play, and management. This shows that the platform does not treat a purchase as an isolated commercial event, but as the beginning of a longer-term ownership relationship between the player and the digital product.

From a system perspective, the transaction record and the personal library serve different business purposes:

- the transaction records the commercial event at a specific point in time
- the library represents the player's ongoing ownership of digital content

This distinction is important because it implies that post-purchase data must be managed separately from purchase data. The library allows the platform to keep track of owned games over time and to support later interactions such as installation, playtime tracking, and access to related content. As a result, Steam demonstrates that digital ownership is not limited to payment confirmation, but extends into a persistent data relationship between the player and the purchased game. From a business analysis perspective, this structure provides a strong foundation for the proposed system. It suggests that ownership-related data should be modeled independently from transaction data, while still remaining linked to the purchasing process. Therefore, Steam serves as a useful reference for understanding how digital products can move from purchase to long-term library management within an integrated platform.

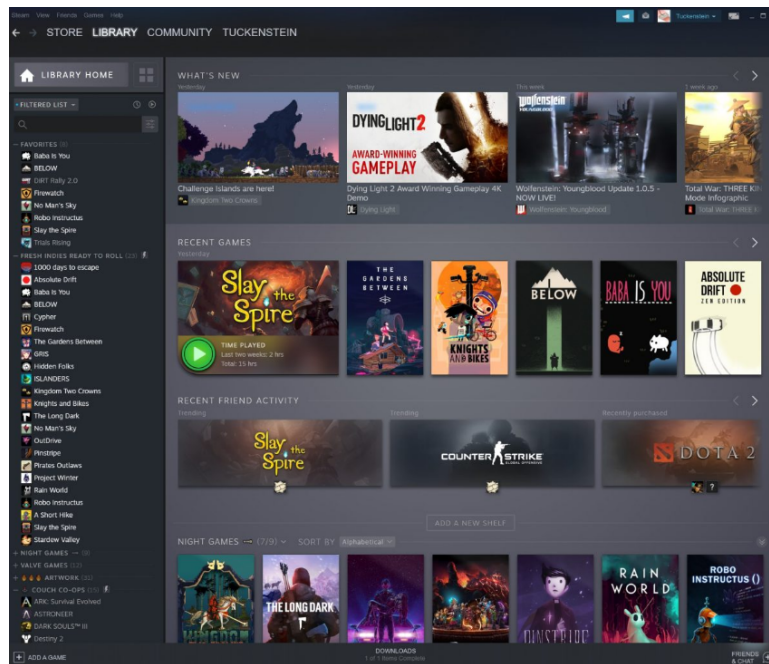


Figure 1.6: Steam personal library interface

The personal library interface shows that purchased games are managed as persistent owned content after the transaction has been completed. This indicates that Steam treats digital ownership as an ongoing relationship between the player and the game rather than a one-time purchase event.

Achievements, Engagement, and Post-Usage Data Steam also shows that the relationship between the player and the game continues after ownership has been established. Once a game is available in the personal library, the platform can track post-usage data that reflects how the player interacts with the product over time. This includes indicators such as playtime, recent activity, and achievement progress. Such data shows that Steam manages not only digital ownership, but also the player's ongoing engagement with purchased content.

From a business perspective, post-usage data serves several important functions:

- it reflects the player's level of interaction with a game
- it supports personalized feedback and progress tracking
- it strengthens long-term engagement with the platform

Among these elements, achievements are especially significant because they represent specific milestones or goals associated with individual games. Their existence implies that the system must track not only which achievements belong to a game, but also which achievements have been unlocked by each player and when this occurred. This shows that Steam maintains gameplay-related records beyond the original purchase and library ownership data.

From a system analysis perspective, this observed structure provides a strong basis for the proposed system. It suggests that engagement-related data should be modeled separately from

both transaction data and basic ownership data, while still remaining connected to the player and the game. Therefore, Steam serves as a useful reference for understanding how a digital platform can manage long-term user interaction, progress, and achievement-related information after purchase.

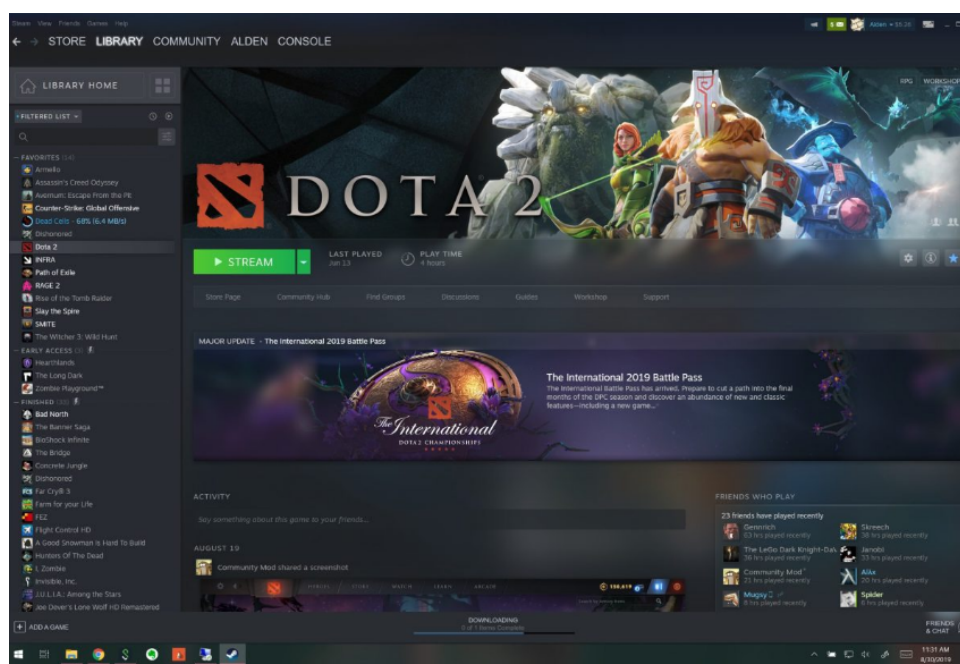


Figure 1.7: Steam game page within the personal library

The interface shows that Steam records post-usage data such as playtime and activity after a game has been added to the user's library.

Community Interaction and User-Generated Content Steam also functions as a community platform in which users do more than purchase and own games. Players can write reviews, express positive or negative opinions, participate in discussions, and interact with other users through community-related features. This shows that the platform manages not only system-generated and publisher-provided data, but also user-generated content that contributes to how games are evaluated and discovered.

From a business perspective, community interaction adds value to the platform in several ways:

- it allows users to share feedback and opinions about games
- it supports social participation beyond purchasing activities
- it influences how other users perceive and discover content

Among these elements, reviews are especially important because they connect a player directly to a specific game through rating behavior and written feedback. This implies that the

system must manage review-related data such as the review content, recommendation outcome, and review date. More broadly, community functions show that Steam treats user participation as an important part of the platform rather than as a secondary feature.

From a system analysis perspective, this observed structure provides a strong basis for the proposed system. It suggests that community-related data should be modeled separately from transaction and ownership data, while still remaining connected to the player and the game. Therefore, Steam serves as a useful reference for understanding how a digital platform can manage user-generated content and social interaction alongside commercial functions.

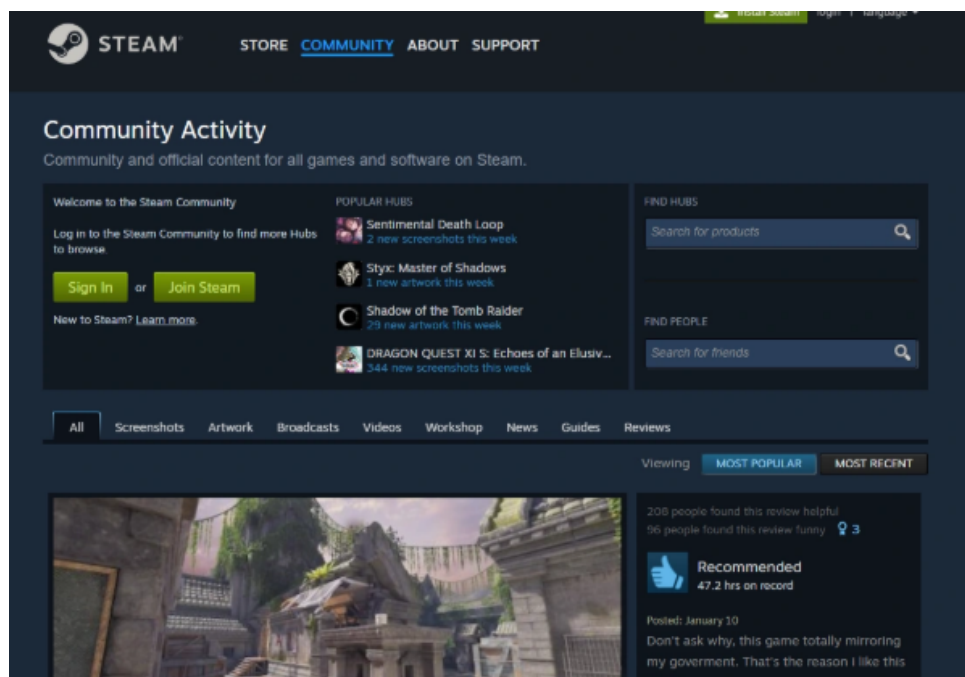


Figure 1.8: Steam community interface

This interface shows that Steam supports community participation through reviews, hubs, and user-generated content such as screenshots and artwork. This indicates that the platform manages social interaction alongside its commercial functions.

Developer/Publisher Participation and Content Lifecycle Steam also reflects the participation of developer and publisher-side actors through the way games are presented, updated, and commercially managed on the platform. Public product pages show developer and publisher information, release details, downloadable content, and promotional structures. This indicates that Steam supports not only consumer-side activities, but also the ongoing lifecycle management of digital products after their initial release.

From a business perspective, developer/publisher participation is important because it connects content creation with platform distribution and long-term commercial activity:

- developers and publishers provide games to the platform

- product information must be maintained and updated over time
- additional content such as DLC can extend the lifecycle of a game
- promotions and pricing changes can influence continued sales performance

This observed structure suggests that a game on Steam is not a static item, but a managed digital product that may continue evolving after release. New content, revised information, and promotional events all indicate that the platform must support publisher-side product maintenance in addition to player-side consumption. As a result, Steam demonstrates that digital distribution platforms must coordinate both the commercial and content lifecycle aspects of each product.

From a system analysis perspective, this provides a strong basis for the proposed system. It suggests that developer/publisher-related data should be modeled separately from player data, while still remaining connected to games and their ongoing lifecycle. Therefore, Steam serves as a useful reference for understanding how publisher-side participation, content extension, and commercial management can be integrated into a digital distribution platform.

Summary Observation

Overall, Steam reflects a two-sided digital platform in which players and developers/publishers are connected through account management, product presentation, purchasing, digital ownership, community participation, and post-release content management. These business functions form an integrated workflow that provides a strong reference foundation for the proposed system described in Section 1.2. Based on this business analysis, the group can identify the main entities, attributes, and relationships of the proposed platform, while the semantic constraints and business rules that cannot be fully represented in the EERD will be presented separately in Section 1.3.

1.2 Proposed system

1.2.1 Description

The proposed system is a comprehensive digital distribution platform and community hub for video game content, conceptually modeled after the industry-leading platform, Steam. Its predominant purpose is to serve as a complete, end-to-end ecosystem that connects content developers with consumers, facilitating not only the publishing and purchase of digital entertainment but also fostering a persistent, social community around those products.

1.2.2 Proposed Application Overview & Users

The system is designed and configured around two primary actors with distinct roles and privileges, satisfying the Two-Sided Marketplace (B2B2C ecosystem) requirement, which concentrates on high-volume, user-friendly experiences, highlighting fast checkouts, personalized marketing, and robust data privacy.

The first actor is the **Player/Consumer**. These are the end-users who engage with the platform to discover new games and new genres, manage their personal collections, and bond socially, either through friendly conversation or constructive criticism. Their functions extend beyond simple purchasing to include building a persistent digital library, tracking in-game achievements, writing reviews, and fostering a network of friendships.

The second actor is the **Developer/Publisher**. These are the content creators or rights holders who utilize the platform as a sales and marketing channel. Their responsibilities consist of the initial listing of their games, delineating what the games entail in brief descriptions, managing post-launch content like downloadable content (DLC) or new seasonal rewards, updating game details, and ultimately tracking the sales performance of their titles.

The primary functions of the system can be grouped into five key areas. **User Management** handles the creation and maintenance of accounts for both Players and Developers, securing their access and storing their profile information. The **Storefront & Purchasing** function is the commercial heart of the platform, allowing users to browse a compartmentalized catalog, manage a temporary shopping cart, and conduct financial transactions. **Library Management** is the persistent, personal space for each player, accurately tracking which specific games and DLCs they own, along with engagement metrics like total playtime. The **Social & Community** function builds a layer of interaction on top of the catalog, enabling players to form friend connections, contribute reviews with ratings, comments, and constructive criticisms. In addition, the function allows players to track their progress in unlocking game-specific achievements, as well as view what achievements they have accomplished in specific games. Finally, **Publisher Functions** provide the backend tools for Developers to upload new products, manage game details and pricing, and create and associate DLC or seasonal updates like battle

passes with their base games.

1.2.3 Entity Types, Attributes, and Relationships

1.2.3.1 Entity Types & Attributes:

The system's data model is centered on a hierarchy of user accounts, with **USER** acting as a superclass. This entity captures the fundamental attributes common to all accounts, including a unique UserID as its primary key, a chosen Username, a verified Email address, a secure PasswordHash, and a JoinDate marking their registration.

From this superclass, two distinct subclasses are specialized. The **PLAYER** subclass inherits all **USER** attributes and adds those specific to a consumer. The attributes of which comprise a WalletBalance, which represents the user's stored funds and is a dynamic attribute updated with each transaction, and a ProfileBio, a text field for personal expression on their profile page. The **DEVELOPER** subclass, conversely, inherits the core **USER** attributes but is augmented with business-oriented information. This includes a registered LegalName—accommodating both individual creators and corporate entities. Furthermore, it incorporates a BankAccount for receiving sales revenue, and importantly, a multivalued attribute, ContactPhone, since a developer may maintain multiple contact numbers for miscellaneous reasons like customer support or business development.

Perhaps, the most central product entity in Steam is, without a shadow of a doubt, **GAME**. It is defined by a unique GameID and several core descriptive attributes such as its Title, a BasePrice, and a ReleaseDate. It also includes a derived attribute, EffectivePrice, which is dynamically calculated at any given time by applying active discounts from associated promotions to the BasePrice. Its description is managed through a composite attribute, SystemRequirements, which is itself composed of discrete elements like OS, Processor, Memory, and Graphics.

To properly facilitate game discovery, the system utilizes a classification entity: **GENRE**. **GENRE** is a strong entity representing the official, broad categories defined by the developer. It is uniquely identified by a GenreName and includes a brief Description. Because a single title can have the characteristics of multiple categories, a game is often linked to several genres; a survival horror title like *Resident Evil Requiem* (2026) would be officially linked to both the "Action" and "Horror" genres, for instance.

Supporting the core game are several dependent entities. **DLC** (Downloadable Content) is a weak entity, as its existence is entirely dependent on a specific base game and whether or not said base game performs well enough in sales to justify the development and release of a DLC. Its identity is partially determined by its relationship to a GameID and is completed by its partial key, DLC_Name. It also has its own Price and ReleaseDate. Similarly, **ACHIEVEMENT** is a weak entity dependent on a **GAME**. It represents in-game challenges or milestones and is identified by its partial key, AchievementName, its Description, and also includes a descriptive

Points value.

REVIEW is another weak entity that exists only in the context of a **GAME**. It is identified by a partial key, ReviewID, and contains the user's feedback, which includes a binary Rating (representing a "Recommended" or "Not Recommended" thumbs-up/down), a detailed text Comment, and the ReviewDate.

Next, the **TRANSACTION** entity is a strong entity representing a financial order. It is uniquely identified by a TransactionID and records the details of a purchase, including the Date when the game was initially purchased, the TotalAmount paid for that specific game, and the PaymentMethod utilized to make said transaction.

Another entity related to the purchasing workflow is the **CART** entity, which acts as a temporary staging area for players before they finalize a purchase. It is uniquely identified by a CartID and includes a LastUpdated timestamp to track its recent activity and potentially recover abandoned carts. Unlike a permanent transaction record, a cart represents a player's current shopping intent, holding selected games in a temporary queue until checkout. At the moment of purchase, the system transfers these contents into a formal, permanent transaction record and subsequently empties the cart so it can be reused for future shopping sessions.

Last but not least, the **PROMOTION** entity is a strong entity designed to handle dynamic pricing and storefront sales events, such as an annual summer sale or a specific publisher weekend. It is uniquely identified by a PromotionID and contains core attributes, consisting of the event's Name, the specific DiscountPercentage being applied, as well as a StartDate and EndDate to automatically manage the lifecycle of the sale. Games are linked to these promotions over time, allowing the platform to offer temporary price reductions to consumers without permanently altering a game's standard base price.

1.2.3.2 Relationships:

The connections between these entities define the platform's logic. A **DEVELOPER** Publishes zero-to-many **GAMES**, but each **GAME** is published by exactly one **DEVELOPER**, thus establishing a one-to-many (1:N) relationship.

The weak entities are tied to their owners through identifying relationships. A **GAME** Has many **DLC** items, and because DLC is a weak entity as it cannot exist without the base game, this is an identifying 1:N relationship. The same pattern applies to **ACHIEVEMENT**; a **GAME** Has **ACHIEVEMENT**, forming an identifying 1:N relationship where the child cannot exist without the parent.

User actions are captured through several relationships. A **PLAYER** Writes many **REVIEWS**, and because **REVIEW** is a weak entity dependent on the player, this forms an identifying 1:N relationship. A **PLAYER** also Makes many **TRANSACTIONS**, with each transaction belonging to a single player, forming a regular 1:N relationship. Furthermore, a **PLAYER** Has a **CART** in a strict one-to-one (1:1) relationship, where the **CART** has total participation as it cannot exist without belonging to exactly one specific player.

The commercial and social cores are modeled with many-to-many (N:M) relationships. The purchase of games is captured by the Contains relationship between **TRANSACTION** and **GAME**. A transaction can contain multiple games, and a game can appear in many transactions. This relationship requires its own attribute, PriceAtPurchase, to accurately record the historical sale price independent of the game's current BasePrice. Before that purchase is finalized, the Queues N:M relationship links a **CART** to a **GAME**, representing the temporary intent to buy those specific titles. This relationship resolves into an associative entity that tracks which games are currently awaiting purchase in a given cart. Similarly, dynamic pricing is handled by the Applies_To N:M relationship between **PROMOTION** and **GAME**, meaning a game can participate in multiple sales events over time, and a specific sale features multiple games. This allows the system to calculate the EffectivePrice of a game at any given time based on active promotions.

The player's library is modeled by the Owns_Library relationship between **PLAYER** and **GAME**. This N:M relationship tracks the games a player owns and is detailed with the PurchaseDate and the dynamically updating TotalPlaytime. Player achievements are tracked via the Unlocks N:M relationship between **PLAYER** and **ACHIEVEMENT**, which includes the UnlockDate attribute.

Finally, the social graph is represented by the recursive N:M relationship, **Friends_With**, on the **PLAYER** entity. This self-referential relationship allows a player to be friends with many other players and includes the DateAdded to remember when the friendship was formed.

1.3 Semantic Constraints Not Fully Represented in the EERD

1. The attribute **Rating** of **REVIEW** must take only one of two binary values: **Recommended** or **Not Recommended**.
2. A **PLAYER** may submit **at most one review** for the same **GAME**.
3. A **PLAYER** may write a review for a **GAME** only if that game already exists in the player's account or library, for example through purchase, key activation, or free acquisition.
4. The attribute **WalletBalance** of **PLAYER** must always be **greater than or equal to 0**.
5. The attributes **BasePrice** of **GAME** and **Price** of **DLC** must always be **greater than or equal to 0**. A value of 0 is allowed for free-to-play or free content.
6. The attribute **DiscountPercentage** of **PROMOTION** must always satisfy the condition:

$$0 < \text{DiscountPercentage} \leq 100$$



7. In **PROMOTION**, **EndDate** must always be later than **StartDate**.
8. A **PLAYER** cannot add a **GAME** to **CART** if that game already exists in the player's library through the **Owns_Library** relationship.
9. The attribute **TotalAmount** of a **TRANSACTION** must equal the sum of all **PriceAtPurchase** values of the games contained in that transaction.
10. **PurchaseDate** in **Owns_Library** and **DateAdded** in **Friends_With** must be on or after the corresponding user's **JoinDate**.
11. An achievement may be unlocked only if the corresponding player already owns the related game. Therefore, **UnlockDate** in **Unlocks** must be on or after the player's **PurchaseDate** for that game.
12. A **PLAYER** can unlock an **ACHIEVEMENT** only if that achievement belongs to the same **GAME** that the player owns.
13. A **PLAYER** should have at most one active cart at any point in time, consistent with the one-to-one relationship between **PLAYER** and **CART** in the proposed design.
14. A promotion can affect the sale price of a game only during its valid time interval. Therefore, a discounted price may be applied in a transaction only if the transaction date falls between the promotion's **StartDate** and **EndDate**.
15. A **DLC** may only be purchased or accessed if the corresponding base **GAME** is already owned by the player.

1.4 Full System Specification

This section presents a consolidated textual specification of the Steam Store Management system. It synthesizes all entities, attributes, relationships, and business rules previously identified in Sections 1.1, 1.2, and 1.3 into a single description. The purpose of this specification is to give a complete picture of the proposed system so that the EERD in Section 2 and the relational schema in Section 3 can be derived and validated against it.

The proposed system is a digital distribution platform inspired by Steam. It must store the entire information of users, games, downloadable content, promotions, transactions, ownership records, reviews, achievements, friendships, and shopping carts. The system supports two main types of users: **players**, who consume digital content, and **developers/publishers**, who provide and manage digital products. Both share a common base entity **USER** that records account-level information such as a unique user identifier, a unique username, a unique email

address, a hashed password, and a join date. The specialization between PLAYER and DEVELOPER is total and disjoint: every user must be either a player or a developer, but not both.

A PLAYER extends the base user with player-specific attributes such as a wallet balance and a personal profile bio. The wallet balance must always be greater than or equal to zero and is reduced when the player completes a purchase. A player may own multiple games through the OWNS_LIBRARY relationship and multiple downloadable contents through the OWNS_DLC relationship. Players may also become friends with other players through the symmetric FRIENDS relationship, in which the smaller user identifier must always be stored before the larger to avoid duplicate friendship records. Each player may unlock achievements that belong to specific games, and the unlock event is recorded together with the achievement, the player, and the time of unlocking through the PLAYER_ACHIEVEMENT relationship.

A DEVELOPER extends the base user with a legal name and an optional bank account used for revenue settlement. Each developer may register one or more contact phone numbers, which are stored in the multi-valued attribute table CONTACT_PHONE. A developer publishes one or more GAME products. Each game is identified by a unique game identifier and includes attributes such as title, base price, release date, system requirements (graphics, operating system, processor, memory), and a derived attribute `effective_price` that reflects the current selling price after applying the best valid promotion. A game may belong to one or more genres through the GAME_GENRE relationship and may be linked to one or more promotions through the GAME_PROMOTION relationship.

The system also manages **downloadable content** (DLC) as a weak entity that depends on a parent GAME. Each DLC is identified by a composite key consisting of the parent game identifier and a local DLC identifier. A DLC has its own title, price, and release date. DLCs may be purchased separately from the base game and are tracked independently in the OWNS_DLC relationship.

Promotions are first-class entities identified by a unique promotion identifier and include attributes such as discount percentage, start date, and end date. The end date must always be greater than or equal to the start date. A promotion may apply to multiple games and a game may receive multiple promotions over time, but at any given moment only the best valid promotion is reflected in the `effective_price` of a game.

Commercial activity is divided into two stages: temporary intent and confirmed transaction. The temporary intent is represented by the CART entity, in which a player places games and DLCs that they intend to purchase later. Cart contents are stored through the QUEUE relationship and may be removed without creating a permanent record. The confirmed transaction is represented by the TRANSACTION entity, which is identified by a transaction identifier and includes attributes such as the buyer player, the transaction date, the payment method, and the total amount. Games purchased in a transaction are recorded in the CONTAINS rela-



tionship together with the derived attribute `price_at_purchase`, which preserves the exact price paid at the time of purchase regardless of any later price change. DLC purchases inside a transaction are recorded in the `BUY_DLC` relationship.

Community interaction is supported through the `REVIEW` entity. A player may write at most one review for the same game, and only for games that the player already owns. Each review records a binary rating that takes only one of two values: `Recommended` or `Not Recommended`, together with the review text and the review date. Reviews are connected to both the player and the game through the `REVIEW` entity itself.

Achievements are managed through the `ACHIEVEMENT` weak entity, identified by the parent game and a local achievement identifier. Each achievement has its own name and description. Achievements unlocked by players are tracked through the `PLAYER_ACHIEVEMENT` relationship together with the unlock time.

The system is also subject to several **semantic constraints** that cannot be fully captured in the EERD and must be enforced at the database level through `CHECK` constraints, triggers, or stored procedure validations. The most important constraints include: the `wallet_balance` of a player must remain non-negative; the `Rating` of a review must be either `Recommended` or `Not Recommended`; a player may submit at most one review per game; a player may write a review only for games they own; promotion dates must be valid ($\text{end_date} \geq \text{start_date}$); transaction totals must equal the sum of `price_at_purchase` of all contained items; and the derived attributes `effective_price` and `price_at_purchase` must be automatically computed and maintained whenever related data changes.

Overall, the specification above defines a digital distribution platform that integrates account management, product publication, promotion-aware purchasing, persistent ownership, community-driven reviews, and achievement-based engagement. This consolidated description serves as the conceptual basis for the EERD presented in Section 2 and the relational mapping presented in Section 3.

2 EERD design

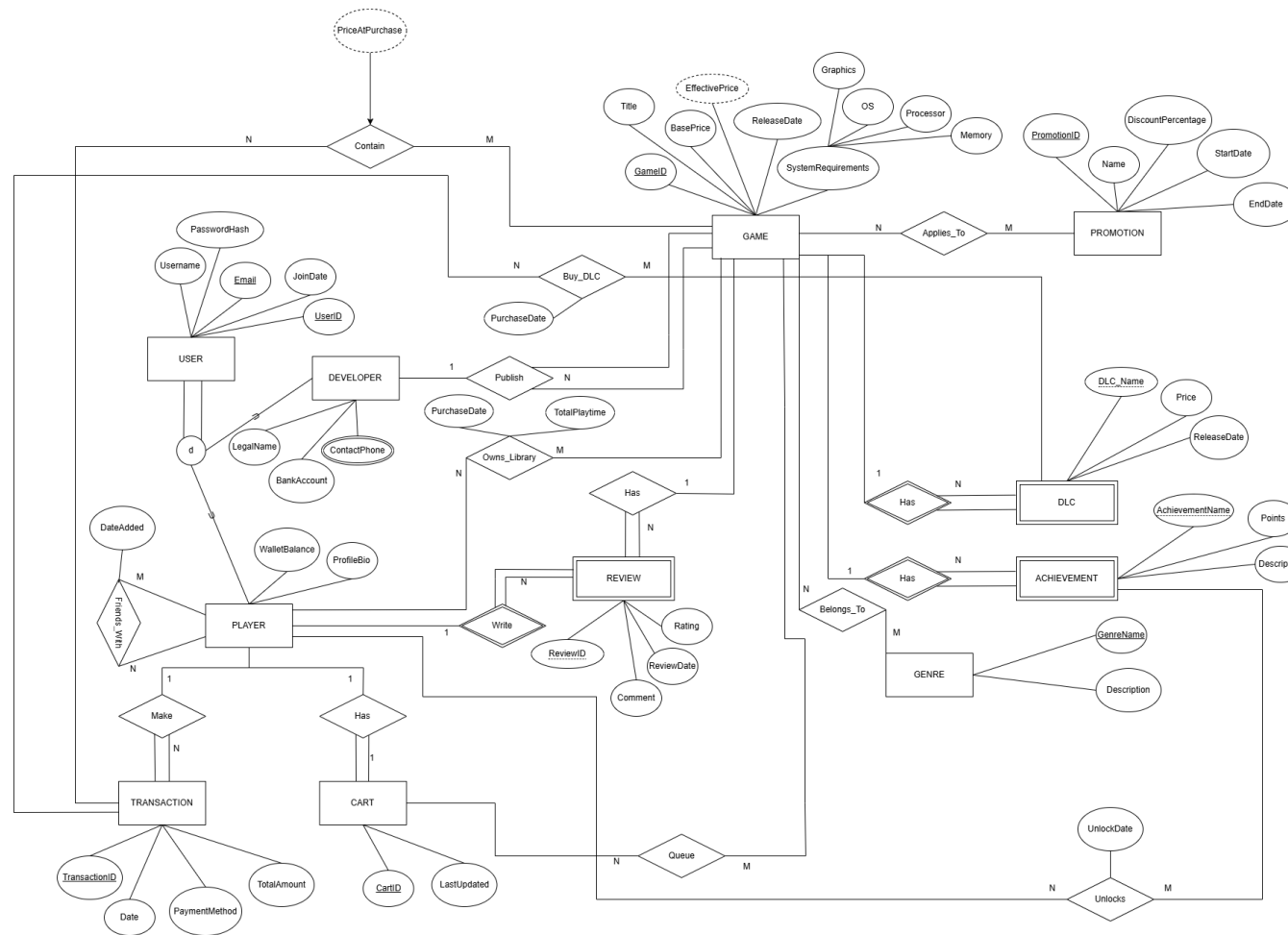


Figure 2.1: Final EERD of the proposed system

The EERD presents the main entities, attributes, and relationships of the proposed system, including superclass/subclass structure, weak entities, and different relationship cardinalities derived from the data requirements analysis.



3 Mapping

User (userID, username, email, passwordHash, joinDate)

- Primary key: userID
- Candidate key: email
- Check total specialization of User.userID in Player, Developer

Player (userID, walletBalance, profileBio)

- Primary key: userID
- Foreign key: userID to User.userID

Developer (userID, legalName, bankAccount)

- Primary key: userID
- Foreign key: userID to User.userID

Promotion (promotionID, name, discountPercentage, startDate, endDate)

- Primary key: promotionID

Genre (genreName, description)

- Primary key: genreName

DLC (gameID, DLC_Name, price, releaseDate, game_ID)

- Primary key: gameID, DLC_Name
- Foreign key: gameID to Game.gameID, game_ID to Game.gameID
- Not null: game_ID

Achievement (gameID, achievementName, points, description, game_ID)

- Primary key: gameID, achievementName
- Foreign key: gameID to Game.gameID, game_ID to Game.gameID
- Not null: game_ID

Review (player_UserID, reviewID, rating, reviewDate, comment, gameID, PlayerID)

- Primary key: player_UserID, reviewID



- Foreign key: gameID to Game.gameID, player_UserID to Player.userID, PlayerID to Player.userID
- Not null: playerID, GameID, player_UserID

Cart (cartID, lastUpdated, userID)

- Primary key: cartID
- Foreign key: userID to Player.userID

Game (gameID, title, basePrice, releaseDate, os, processor, memory, graphics, developer_UserID)

- Primary key: gameID
- Foreign key: developer_UserID to Developer.userID
- Not null: developer_UserID

Transaction (TransactionID, date, paymentMethod, totalAmount, player_UserID)

- Primary key: TransactionID
- Foreign key: player_UserID to Player.userID
- Not null: player_UserID

Contain (TransactionID, gameID)

- Primary key: TransactionID, gameID
- Foreign key: TransactionID to Transaction.TransactionID, gameID to Game.gameID

Buy_DLC (TransactionID, gameID, DLC_Name, PurchaseDate)

- Primary key: TransactionID, gameID, DLC_Name
- Foreign key: TransactionID to Transaction.TransactionID, (gameID, DLC_Name) to DLC.(gameID, DLC_Name)

Owns_Library (player_UserID, gameID, purchaseDate, totalPlaytime)

- Primary key: player_UserID, gameID
- Foreign key: player_UserID to Player.userID, gameID to Game.gameID

Belongs_To (gameID, genreName)

- Primary key: gameID, genreName



- Foreign key: gameID to Game.gameID, genreName to Genre.genreName

Applies_To (gameID, promotionID)

- Primary key: gameID, promotionID
- Foreign key: gameID to Game.gameID, promotionID to Promotion.promotionID

Queue (cartID, gameID)

- Primary key: cartID, gameID
- Foreign key: cartID to Cart.cartID, gameID to Game.gameID

Friends_With (player_UserID_1, player_UserID_2, dateAdded)

- Primary key: player_UserID_1, player_UserID_2
- Foreign key: player_UserID_1 to Player.userID, player_UserID_2 to Player.userID

Unlocks (player_UserID, gameID, achievementName, unlockDate)

- Primary key: player_UserID, gameID, achievementName
- Foreign key: player_UserID to Player.userID, (gameID, achievementName) to Achievement.(gameID, achievementName)

Developer_Phone (developer_UserID, contactPhone)

- Primary key: developer_UserID, contactPhone
- Foreign key: developer_UserID to Developer.userID

4 Database Implementation

4.1 Implementing Tables and Integrity Constraints

This section presents the physical implementation of the relational schema in MySQL. The detailed relations, primary keys, and foreign keys were already described in the Mapping section. Therefore, this section focuses on how the mapped relations were translated into executable SQL table definitions and integrity constraints. The database is implemented under the name steam. The implementation includes entity tables, weak entity tables, relationship tables, and associative tables required by the Steam Store Management system.

4.1.1 Full SQL Implementation

The following SQL script shows the complete table creation implementation used in the project.

Listing 4.1: Full SQL script for table creation and integrity constraints

```
1 CREATE TABLE `USER` (  
2     user_id INT AUTO_INCREMENT PRIMARY KEY,  
3     username VARCHAR(50) NOT NULL UNIQUE,  
4     email VARCHAR(100) NOT NULL UNIQUE,  
5     password_hash VARCHAR(255) NOT NULL,  
6     join_date DATE NOT NULL  
7 );  
8  
9 CREATE TABLE `PLAYER` (  
10     user_id INT PRIMARY KEY,  
11     wallet_balance DECIMAL(10,2) DEFAULT 0 CHECK (wallet_balance >=  
12         0),  
13     profile_bio TEXT,  
14     FOREIGN KEY (user_id) REFERENCES `USER` (user_id)  
15 );  
16  
17 CREATE TABLE `DEVELOPER` (  
18     user_id INT PRIMARY KEY,  
19     legal_name VARCHAR(100) NOT NULL,  
20     bank_account VARCHAR(100),  
21     FOREIGN KEY (user_id) REFERENCES `USER` (user_id)  
22 );
```

```
23 CREATE TABLE `CONTACT_PHONE` (  
24     developer_user_id INT NOT NULL,  
25     contact_phone VARCHAR(20) NOT NULL,  
26     PRIMARY KEY (developer_user_id, contact_phone),  
27     FOREIGN KEY (developer_user_id) REFERENCES `DEVELOPER` (user_id)  
28     ON DELETE CASCADE  
29 );  
30  
31 CREATE TABLE `GAME` (  
32     game_id INT AUTO_INCREMENT PRIMARY KEY,  
33     title VARCHAR(100) NOT NULL,  
34     base_price DECIMAL(10,2) NOT NULL CHECK (base_price >= 0),  
35     release_date DATE,  
36     graphics VARCHAR(50),  
37     os VARCHAR(50),  
38     processor VARCHAR(50),  
39     memory VARCHAR(50),  
40     developer_id INT NOT NULL,  
41     FOREIGN KEY (developer_id) REFERENCES `DEVELOPER` (user_id)  
42 );  
43  
44 CREATE TABLE `PROMOTION` (  
45     promotion_id INT AUTO_INCREMENT PRIMARY KEY,  
46     name VARCHAR(100) NOT NULL,  
47     discount_percentage DECIMAL(5,2) CHECK (discount_percentage > 0  
48     AND discount_percentage <= 100),  
49     start_date DATE,  
50     end_date DATE,  
51     CHECK (end_date > start_date)  
52 );  
53  
54 CREATE TABLE `GENRE` (  
55     genre_name VARCHAR(50) NOT NULL UNIQUE PRIMARY KEY,  
56     description TEXT  
57 );  
58  
59 CREATE TABLE `DLC` (  
60     game_id INT NOT NULL,
```



```
59     dlc_name VARCHAR(100) NOT NULL,
60     price DECIMAL(10,2) CHECK (price >= 0),
61     release_date DATE,
62     PRIMARY KEY (game_id, dlc_name),
63     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id)
64 );
65
66 CREATE TABLE `ACHIEVEMENT` (
67     game_id INT NOT NULL,
68     achievement_name VARCHAR(100) NOT NULL,
69     points INT CHECK (points >= 0),
70     description TEXT,
71     PRIMARY KEY (game_id, achievement_name),
72     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id)
73 );
74
75 CREATE TABLE `REVIEW` (
76     review_id INT AUTO_INCREMENT PRIMARY KEY,
77     rating VARCHAR(20) NOT NULL CHECK (rating IN ('Recommended', '
78         Not Recommended')),
79     comment TEXT,
80     review_date DATE,
81     player_id INT NOT NULL,
82     game_id INT NOT NULL,
83     UNIQUE (player_id, game_id),
84     FOREIGN KEY (player_id) REFERENCES `PLAYER`(user_id),
85     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id)
86 );
87
88 CREATE TABLE `CART` (
89     cart_id INT AUTO_INCREMENT PRIMARY KEY,
90     last_updated DATETIME,
91     player_id INT UNIQUE,
92     FOREIGN KEY (player_id) REFERENCES `PLAYER`(user_id)
93 );
94
95 CREATE TABLE `TRANSACTION` (
96     transaction_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
96     date DATETIME NOT NULL,
97     payment_method VARCHAR(50),
98     total_amount DECIMAL(10,2) CHECK (total_amount >= 0),
99     player_id INT NOT NULL,
100     FOREIGN KEY (player_id) REFERENCES `PLAYER`(user_id)
101 );
102
103 CREATE TABLE `FRIENDS` (
104     player_id INT,
105     friend_id INT,
106     PRIMARY KEY (player_id, friend_id),
107     FOREIGN KEY (player_id) REFERENCES `PLAYER`(user_id) ON DELETE
108         CASCADE,
109     FOREIGN KEY (friend_id) REFERENCES `PLAYER`(user_id) ON DELETE
110         CASCADE,
111     CHECK (player_id <> friend_id)
112 );
113
114 CREATE TABLE `OWNS_LIBRARY` (
115     player_id INT,
116     game_id INT,
117     purchase_date DATE,
118     total_playtime INT DEFAULT 0 CHECK (total_playtime >= 0),
119     PRIMARY KEY (player_id, game_id),
120     FOREIGN KEY (player_id) REFERENCES `PLAYER`(user_id) ON DELETE
121         CASCADE,
122     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id) ON DELETE
123         CASCADE
124 );
125
126 CREATE TABLE `CONTAINS` (
127     transaction_id INT,
128     game_id INT,
129     price_at_purchase DECIMAL(10,2) CHECK (price_at_purchase >= 0),
130     PRIMARY KEY (transaction_id, game_id),
131     FOREIGN KEY (transaction_id) REFERENCES `TRANSACTION`(
132         transaction_id),
133     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id)
```



```
129 );
130
131 CREATE TABLE `GAME_PROMOTION` (
132     game_id INT,
133     promotion_id INT,
134     PRIMARY KEY (game_id, promotion_id),
135     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id),
136     FOREIGN KEY (promotion_id) REFERENCES `PROMOTION`(promotion_id)
137 );
138
139 CREATE TABLE `GAME_GENRE` (
140     game_id INT,
141     genre_name VARCHAR(50),
142     PRIMARY KEY (game_id, genre_name),
143     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id),
144     FOREIGN KEY (genre_name) REFERENCES `GENRE`(genre_name)
145 );
146
147 CREATE TABLE `QUEUE` (
148     cart_id INT,
149     game_id INT,
150     PRIMARY KEY (cart_id, game_id),
151     FOREIGN KEY (cart_id) REFERENCES `CART`(cart_id),
152     FOREIGN KEY (game_id) REFERENCES `GAME`(game_id)
153 );
154
155 CREATE TABLE `PLAYER_ACHIEVEMENT` (
156     player_id INT,
157     game_id INT,
158     achievement_name VARCHAR(100),
159     unlock_date DATE,
160     PRIMARY KEY (player_id, game_id, achievement_name),
161     FOREIGN KEY (player_id) REFERENCES `PLAYER`(user_id),
162     FOREIGN KEY (game_id, achievement_name) REFERENCES `ACHIEVEMENT`
163         `(game_id, achievement_name)`
164 );
165 CREATE TABLE `BUY_DLC` (
```



```
166     transaction_id INT,  
167     game_id INT,  
168     dlc_name VARCHAR(100),  
169     purchase_date DATE,  
170     PRIMARY KEY (transaction_id, game_id, dlc_name),  
171     FOREIGN KEY (transaction_id) REFERENCES `TRANSACTION` (  
        transaction_id) ON DELETE CASCADE,  
172     FOREIGN KEY (game_id, dlc_name) REFERENCES `DLC` (game_id,  
        dlc_name) ON DELETE CASCADE  
173 );  
174  
175 CREATE TABLE `OWNS_DLC` (  
176     player_id INT,  
177     game_id INT,  
178     dlc_name VARCHAR(100),  
179     purchase_date DATE,  
180     PRIMARY KEY (player_id, game_id, dlc_name),  
181     FOREIGN KEY (player_id) REFERENCES `PLAYER` (user_id) ON DELETE  
        CASCADE,  
182     FOREIGN KEY (game_id, dlc_name) REFERENCES `DLC` (game_id,  
        dlc_name) ON DELETE CASCADE  
183 );
```

4.1.2 Execution Result

After executing the schema script, all tables were successfully created in the steam_store database. The created schema reflects the relational mapping from the previous design phase while enforcing important integrity constraints directly at the database level.

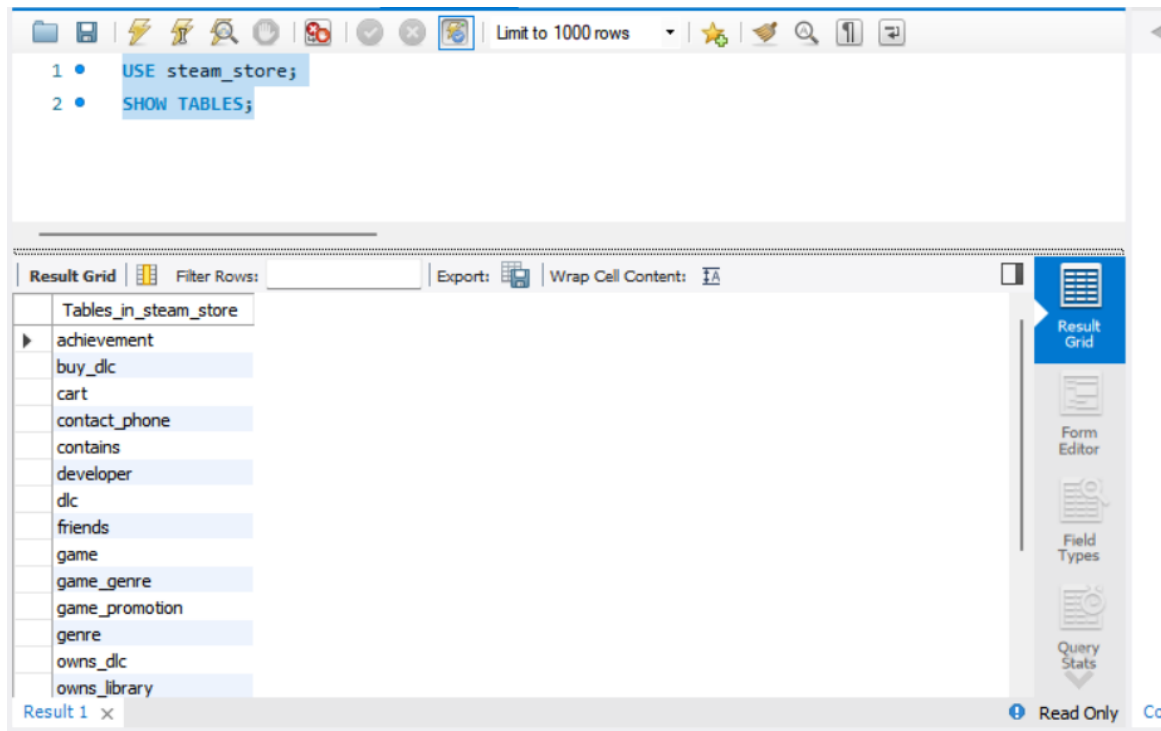


Figure 4.1: Successful creation of database tables in MySQL Workbench

4.2 Sample Data Insertion

This section presents the consolidated sample data insertion script used by the group. The script combines the original sample data and the extended sample data so that all members can initialize and test the database with the same dataset.

The insertion script covers all tables in the Steam Store Management database, including users, players, developers, games, DLCs, promotions, transactions, ownership records, reviews, carts, achievements, and relationship tables. To avoid foreign key order issues during the insertion process, foreign key checks are temporarily disabled before inserting data and enabled again after all records have been inserted.

4.2.1 Sample Data Coverage

The inserted data represents realistic operations of a Steam-like digital distribution platform. The dataset includes player accounts, developer accounts, game products, downloadable content, promotions, purchase transactions, game ownership, DLC ownership, player reviews, carts, friends, and achievement records.

Table 4.1: Summary of inserted sample data

Table	Rows	Description
USER	18	User accounts including players and developers
PLAYER	10	Player profiles and wallet balances
DEVELOPER	8	Developer and publisher information
CONTACT_PHONE	8	Developer contact phone numbers
GENRE	5	Game genre categories
PROMOTION	7	Discount campaigns and sale events
GAME	11	Game products published by developers
DLC	10	Downloadable content linked to games
ACHIEVEMENT	5	Game achievement records
REVIEW	10	Player reviews for games
CART	5	Player shopping carts
TRANSACTION	15	Completed purchase transactions
FRIENDS	5	Friendship relationships between players
OWNS_LIBRARY	5	Player game ownership records
CONTAINS	17	Games included in purchase transactions
GAME_PROMOTION	5	Game and promotion relationships
GAME_GENRE	13	Game and genre relationships
QUEUE	5	Games temporarily stored in carts
PLAYER_ACHIEVEMENT	5	Achievements unlocked by players
BUY_DLC	5	DLC purchases in transactions
OWNS_DLC	5	DLC ownership records

4.2.2 Data Design Explanation

The sample data is designed to support later testing of stored procedures, triggers, functions, and application interfaces. For example, the inserted users are divided into players and developers. Each game is linked to a developer, while DLC records are linked to their corre-

sponding base games. Transactions are connected to players, and purchased games are recorded through the CONTAINS table. DLC purchases are represented through BUY_DLC, while long-term ownership is represented through OWNS_LIBRARY and OWNS_DLC.

This dataset is meaningful because it does not only insert isolated records. Instead, the inserted records are connected through foreign key relationships and reflect realistic platform operations such as purchasing games, owning DLCs, reviewing games, adding games to carts, applying promotions, and unlocking achievements.

4.2.3 Full SQL Data Insertion Script

The following SQL script is the complete sample data insertion script used by the group.

Listing 4.2: Full SQL script for inserting sample data

```
1
2 SET FOREIGN_KEY_CHECKS = 0;
3
4 -- 1. USER TABLE
5 INSERT INTO `USER` (user_id, username, email, password_hash,
6   join_date) VALUES
7 (1, 'khoitrان', 'khoi.tran@gmail.com', 'hash1', '2023-01-15'),
8 (2, 'huylong', 'long.phan@gmail.com', 'hash2', '2023-02-20'),
9 (3, 'thanhnam', 'nam.pham@gmail.com', 'hash3', '2023-03-10'),
10 (4, 'thienan', 'an.trieu@gmail.com', 'hash4', '2023-04-05'),
11 (5, 'giahung', 'hung.nguyen@gmail.com', 'hash5', '2023-05-12'),
12 (6, 'riotgames', 'contact@riotgames.com', 'dev1', '2020-10-01'),
13 (7, 'cd_projekt', 'hello@cdprojekt.com', 'dev2', '2019-11-11'),
14 (8, 'valve_corp', 'support@valvesoftware.com', 'dev3', '2018-05-20'
15   ),
16 (9, 'mihoyo', 'business@mihoyo.com', 'dev4', '2021-06-15'),
17 (10, 'vng_games', 'admin@vng.com.vn', 'dev5', '2015-08-08'),
18 (11, 'faker_vn', 'faker.vn@gmail.com', 'hash11', '2023-06-01'),
19 (12, 'mixigaming', 'do.mixi@gmail.com', 'hash12', '2023-07-15'),
20 (13, 'crisdevil', 'cris.phan@gmail.com', 'hash13', '2023-08-20'),
21 (14, 'phuongly', 'ly.phuong@gmail.com', 'hash14', '2023-09-10'),
22 (15, 'sontungmtp', 'mtp.ent@gmail.com', 'hash15', '2023-10-05'),
23 (16, 'rockstar_games', 'contact@rockstargames.com', 'dev16', '
24   2015-01-01'),
25 (17, 'square_enix', 'info@square-enix.com', 'dev17', '2016-02-02'),
26 (18, 'capcom_ltd', 'support@capcom.co.jp', 'dev18', '2017-03-03');
```



```
24
25 -- 2. PLAYER TABLE
26 INSERT INTO `PLAYER` (user_id, wallet_balance, profile_bio) VALUES
27 (1, 500000.0, 'Yasuo main, 20 games per day'),
28 (2, 1000000.0, 'Passionate about RPG games'),
29 (3, 50000.0, 'Plays games to relieve stress'),
30 (4, 250000.0, 'FPS sharpshooter'),
31 (5, 0.0, 'Free-to-play grinder, never tops up'),
32 (11, 1500000.0, 'Pro boosting player'),
33 (12, 5000000.0, 'Auto Chess legend'),
34 (13, 2000000.0, 'Casual game reviewer'),
35 (14, 500000.0, 'Plays mostly for fun'),
36 (15, 10000000.0, 'Whale, top-up king');
37
38 -- 3. DEVELOPER TABLE
39 INSERT INTO `DEVELOPER` (user_id, legal_name, bank_account) VALUES
40 (6, 'Riot Games Inc.', 'BANK_123'),
41 (7, 'CD Projekt S.A.', 'BANK_987'),
42 (8, 'Valve Corporation', 'BANK_111'),
43 (9, 'miHoYo Co., Ltd.', 'BANK_444'),
44 (10, 'VNG Corporation', 'BANK_777'),
45 (16, 'Rockstar Games', 'BANK_RSTAR'),
46 (17, 'Square Enix Co., Ltd.', 'BANK_SQEX'),
47 (18, 'Capcom Co., Ltd.', 'BANK_CAPCOM');
48
49 -- 4. CONTACT_PHONE TABLE
50 INSERT INTO `CONTACT_PHONE` (developer_user_id, contact_phone)
51     VALUES
52 (6, '+1-800-555-0101'),
53 (7, '+48-22-123-4567'),
54 (8, '+1-800-555-0202'),
55 (9, '+86-21-1234-5678'),
56 (10, '+84-28-3962-3688'),
57 (16, '+1-800-RSTAR-01'),
58 (17, '+81-3-5292-8100'),
59 (18, '+81-6-6920-3600');
60
61 -- 5. GENRE TABLE
```



```
61 INSERT INTO `GENRE` (genre_name, description) VALUES
62 ('Action', 'Action games'),
63 ('RPG', 'Role-playing games'),
64 ('FPS', 'First-person shooter'),
65 ('MOBA', 'Multiplayer online battle arena'),
66 ('Strategy', 'Strategy games');
67
68 -- 6. PROMOTION TABLE
69 INSERT INTO `PROMOTION` (promotion_id, name, discount_percentage,
70 start_date, end_date) VALUES
71 (1, 'Summer Sale', 50.0, '2026-06-01', '2026-06-30'),
72 (2, 'Winter Sale', 70.0, '2026-12-15', '2026-12-31'),
73 (3, 'Black Friday', 80.0, '2026-11-25', '2026-11-30'),
74 (4, 'Lunar New Year', 30.0, '2026-02-10', '2026-02-20'),
75 (5, 'Weekend Deal', 15.0, '2026-04-18', '2026-04-20'),
76 (6, 'Halloween Sale', 60.0, '2026-10-25', '2026-11-01'),
77 (7, 'Spring Festival', 40.0, '2026-03-15', '2026-03-25');
78
79 -- 7. GAME TABLE
80 INSERT INTO `GAME` (game_id, title, base_price, release_date,
81 graphics, os, processor, memory, developer_id) VALUES
82 (1, 'League of Legends', 0.0, '2009-10-27', 'GTX 1050', 'Windows 10',
83 'Intel i3', '8GB RAM', 6),
84 (2, 'Valorant', 0.0, '2020-06-02', 'GTX 1050 Ti', 'Windows 10', 'Intel i5',
85 '8GB RAM', 6),
86 (3, 'Cyberpunk 2077', 990000.0, '2020-12-10', 'RTX 3060', 'Windows 11',
87 'Intel i7', '16GB RAM', 7),
88 (4, 'Half-Life: Alyx', 1200000.0, '2020-03-23', 'GTX 1060', 'Windows 10',
89 'Intel i5', '12GB RAM', 8),
90 (5, 'Genshin Impact', 0.0, '2020-09-28', 'GTX 1060', 'Windows 10',
91 'Intel i5', '8GB RAM', 9),
92 (6, 'Grand Theft Auto V', 500000.0, '2015-04-14', 'GTX 660', 'Windows 10',
93 'Intel i5', '8GB RAM', 16),
94 (7, 'Red Dead Redemption 2', 1000000.0, '2019-11-05', 'GTX 1060', 'Windows 10',
95 'Intel i7', '12GB RAM', 16),
96 (8, 'Final Fantasy VII Remake', 1500000.0, '2021-12-16', 'RTX 2060', 'Windows 10',
97 'Intel i7', '16GB RAM', 17),
98 (9, 'Resident Evil 4', 1200000.0, '2023-03-24', 'RX 5600 XT', 'Windows 10',
99 'AMD Ryzen 5', '16GB RAM', 18);
```

```
Windows 11', 'Ryzen 5', '16GB RAM', 18),
89 (10, 'Monster Hunter: World', 600000.0, '2018-08-09', 'GTX 1060', '
    Windows 10', 'Intel i5', '8GB RAM', 18),
90 (11, 'The Witcher 3: Wild Hunt', 400000.0, '2015-05-18', 'GTX 770',
    'Windows 10', 'Intel i5', '8GB RAM', 7);
91
92 -- 8. DLC TABLE
93 INSERT INTO `DLC` (game_id, dlc_name, price, release_date) VALUES
94 (3, 'Phantom Liberty', 450000.0, '2023-09-26'),
95 (1, 'Arcane Jinx Skin', 200000.0, '2024-11-01'),
96 (5, 'Moonlight Blessing', 109000.0, '2020-10-01'),
97 (2, 'Prime Vandal Bundle', 300000.0, '2021-05-15'),
98 (4, 'Soundtrack Collection', 150000.0, '2020-04-10'),
99 (6, 'Criminal Enterprise Starter Pack', 200000.0, '2017-12-15'),
100 (8, 'EPISODE INTERmission', 400000.0, '2021-12-16'),
101 (9, 'Separate Ways', 250000.0, '2023-09-21'),
102 (10, 'Iceborne Expansion', 500000.0, '2020-01-09'),
103 (11, 'Blood and Wine', 200000.0, '2016-05-31');
104
105 -- 9. ACHIEVEMENT TABLE
106 INSERT INTO `ACHIEVEMENT` (game_id, achievement_name, points,
    description) VALUES
107 (1, 'Penta Kill', 100, 'Eliminate 5 enemy champions in a row'),
108 (3, 'The Star', 50, 'Complete The Star ending'),
109 (4, 'Gnome Chompski', 200, 'Bring the Gnome statue to the bunker'),
110 (5, 'Abyss Floor 12', 150, 'Clear floor 12 of the Spiral Abyss'),
111 (2, 'Ace', 100, 'Single-handedly eliminate all enemies');
112
113 -- 10. REVIEW TABLE
114 INSERT INTO `REVIEW` (rating, comment, review_date, player_id,
    game_id) VALUES
115 ('Recommended', 'Extremely addictive game!', '2024-01-10', 1, 3),
116 ('Recommended', 'Good game but a bit laggy', '2024-02-12', 2, 1),
117 ('Recommended', 'Top-tier VR experience', '2024-03-15', 3, 4),
118 ('Not Recommended', 'Gacha rates are too brutal', '2024-04-20', 4,
    5),
119 ('Recommended', 'Best tactical shooter, hands down', '2024-05-25',
    5, 2),
```



```
120 ('Recommended', 'Masterpiece open world!', '2024-05-20', 11, 7),
121 ('Recommended', 'Stunning graphics, great storyline', '2024-05-25',
    12, 8),
122 ('Recommended', 'Gwent is more fun than the main quest', '
    2024-06-05', 13, 11),
123 ('Not Recommended', 'Ashley screaming for help is deafening', '
    2024-06-15', 14, 9),
124 ('Recommended', 'Fun grindy game', '2024-07-10', 15, 10);
125
126 -- 11. CART TABLE
127 INSERT INTO `CART` (cart_id, last_updated, player_id) VALUES
128 (1, '2026-04-15 10:00:00', 1),
129 (2, '2026-04-16 11:30:00', 2),
130 (3, '2026-04-17 14:45:00', 3),
131 (4, '2026-04-18 09:15:00', 4),
132 (5, '2026-04-19 20:20:00', 5);
133
134 -- 12. TRANSACTION TABLE
135 INSERT INTO `TRANSACTION` (transaction_id, date, payment_method,
    total_amount, player_id) VALUES
136 (1, '2023-12-25 08:30:00', 'Credit Card', 990000.0, 1),
137 (2, '2024-01-05 14:20:00', 'Momo', 1200000.0, 2),
138 (3, '2024-02-14 19:45:00', 'ZaloPay', 450000.0, 1),
139 (4, '2024-03-08 21:10:00', 'PayPal', 109000.0, 4),
140 (5, '2024-04-01 10:05:00', 'Credit Card', 200000.0, 2),
141 (6, '2024-05-10 10:00:00', 'Momo', 1500000.0, 11),
142 (7, '2024-05-15 14:30:00', 'Credit Card', 2200000.0, 12),
143 (8, '2024-06-01 09:15:00', 'ZaloPay', 600000.0, 13),
144 (9, '2024-06-10 20:45:00', 'PayPal', 1700000.0, 14),
145 (10, '2024-07-05 11:20:00', 'Credit Card', 200000.0, 15),
146 (11, '2024-07-20 16:00:00', 'Momo', 1200000.0, 1),
147 (12, '2024-08-15 08:30:00', 'Momo', 400000.0, 2),
148 (13, '2024-09-01 19:10:00', 'ZaloPay', 1500000.0, 3),
149 (14, '2024-10-31 22:00:00', 'Credit Card', 500000.0, 4),
150 (15, '2024-11-26 23:59:00', 'PayPal', 1000000.0, 5);
151
152 -- 13. FRIENDS TABLE
153 INSERT INTO `FRIENDS` (player_id, friend_id) VALUES
```




```
154 (1, 2), (1, 3), (2, 4), (3, 5), (4, 5);
155
156 -- 14. OWNS_LIBRARY TABLE
157 INSERT INTO `OWNS_LIBRARY` (player_id, game_id, purchase_date,
    total_playtime) VALUES
158 (1, 3, '2023-12-25', 120),
159 (1, 2, '2023-12-25', 500),
160 (2, 4, '2024-01-05', 45),
161 (4, 5, '2024-03-08', 300),
162 (2, 1, '2024-04-01', 1000);
163
164 -- 15. CONTAINS TABLE
165 INSERT INTO `CONTAINS` (transaction_id, game_id, price_at_purchase)
    VALUES
166 (1, 3, 990000.0),
167 (2, 4, 1200000.0),
168 (4, 5, 0.0),
169 (5, 1, 0.0),
170 (1, 2, 0.0),
171 (6, 7, 1000000.0),
172 (6, 6, 500000.0),
173 (7, 8, 1500000.0),
174 (7, 10, 600000.0),
175 (8, 11, 400000.0),
176 (9, 9, 1200000.0),
177 (10, 8, 1500000.0),
178 (11, 9, 1200000.0),
179 (12, 11, 400000.0),
180 (13, 7, 1000000.0),
181 (14, 6, 500000.0),
182 (15, 7, 1000000.0);
183
184 -- 16. GAME_PROMOTION TABLE
185 INSERT INTO `GAME_PROMOTION` (game_id, promotion_id) VALUES
186 (3, 1), (4, 2), (3, 3), (5, 4), (1, 5);
187
188 -- 17. GAME_GENRE TABLE
189 INSERT INTO `GAME_GENRE` (game_id, genre_name) VALUES
```



```
190 (1, 'MOBA'), (2, 'FPS'), (3, 'RPG'), (3, 'Action'), (4, 'FPS'),
191 (6, 'Action'), (7, 'Action'), (7, 'RPG'), (8, 'RPG'), (9, 'Action')
192 ,
193 (10, 'RPG'), (10, 'Action'), (11, 'RPG');
194
195 -- 18. QUEUE TABLE
196 INSERT INTO `QUEUE` (cart_id, game_id) VALUES
197 (1, 3), (1, 4), (2, 5), (3, 1), (5, 2);
198
199 -- 19. PLAYER_ACHIEVEMENT TABLE
200 INSERT INTO `PLAYER_ACHIEVEMENT` (player_id, game_id,
201 achievement_name, unlock_date) VALUES
202 (1, 3, 'The Star', '2024-01-20'),
203 (2, 4, 'Gnome Chompski', '2024-02-10'),
204 (4, 5, 'Abyss Floor 12', '2024-03-25'),
205 (1, 2, 'Ace', '2023-12-30'),
206 (2, 1, 'Penta Kill', '2024-05-01');
207
208 -- 20. BUY_DLC TABLE
209 INSERT INTO `BUY_DLC` (transaction_id, game_id, dlc_name,
210 purchase_date) VALUES
211 (3, 3, 'Phantom Liberty', '2024-02-14'),
212 (5, 1, 'Arcane Jinx Skin', '2024-04-01'),
213 (4, 5, 'Moonlight Blessing', '2024-03-08'),
214 (1, 2, 'Prime Vandal Bundle', '2023-12-25'),
215 (2, 4, 'Soundtrack Collection', '2024-01-05');
216
217 -- 21. OWNS_DLC TABLE
218 INSERT INTO `OWNS_DLC` (player_id, game_id, dlc_name, purchase_date
219 ) VALUES
220 (1, 3, 'Phantom Liberty', '2024-02-14'),
221 (2, 1, 'Arcane Jinx Skin', '2024-04-01'),
222 (4, 5, 'Moonlight Blessing', '2024-03-08'),
223 (1, 2, 'Prime Vandal Bundle', '2023-12-25'),
224 (2, 4, 'Soundtrack Collection', '2024-01-05');
225
226 SET FOREIGN_KEY_CHECKS = 1;
```

4.2.4 Execution Result

After executing the sample data insertion script, the number of inserted records was verified using COUNT (*) queries for all tables. The result in Figure 4.2 shows that all main entity tables and relationship tables contain data and are ready for testing later database operations.

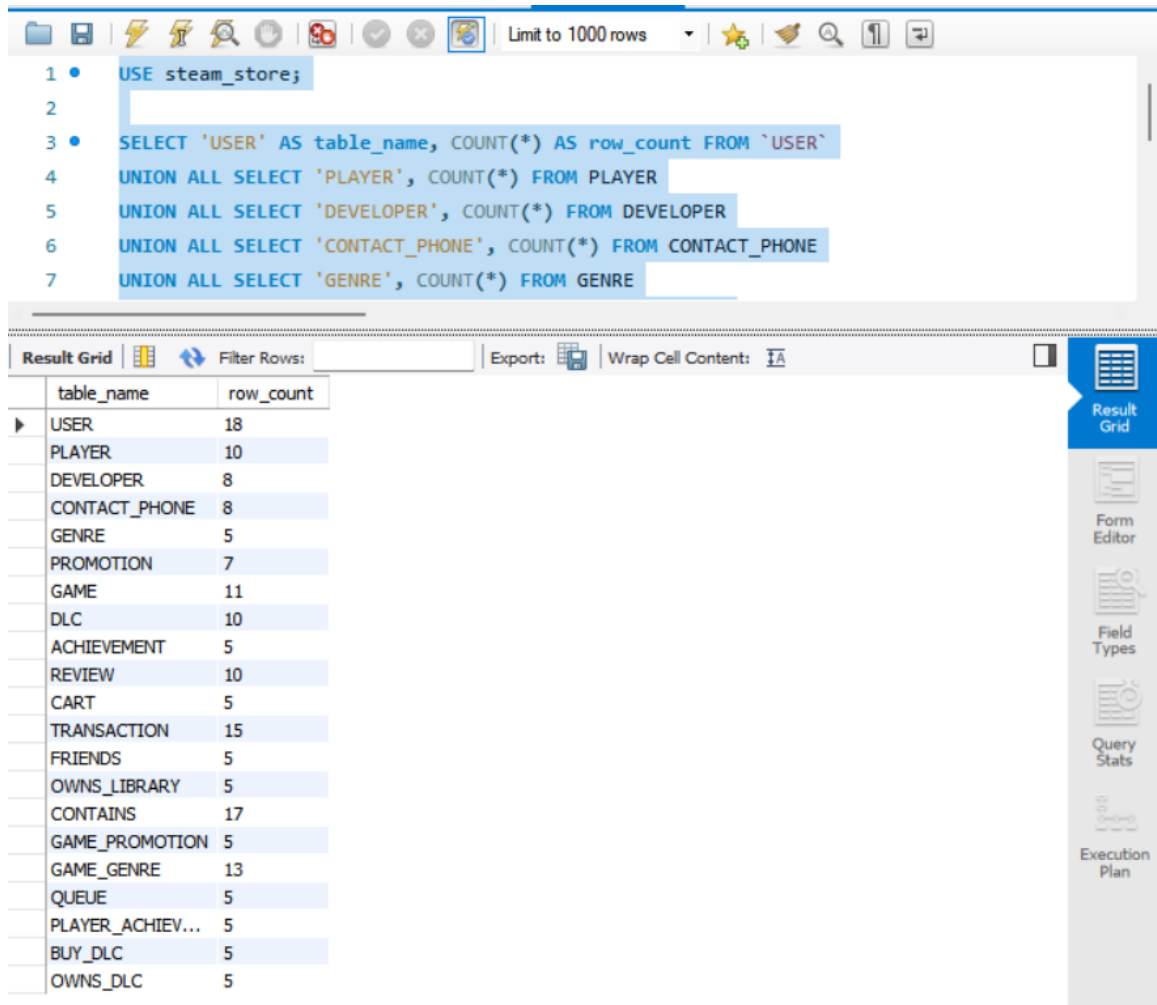


Figure 4.2: Verification of inserted sample data using row count queries

5 Stored Procedures, Triggers, and Functions

5.1 Stored Procedures for Insert, Update, and Delete

This section presents the stored procedures used to perform insert, update, and delete operations on the GAME table. The GAME table was selected because it is one of the central entities of the Steam Store Management system. It stores digital game products that are later connected to developers, genres, DLCs, promotions, transactions, ownership records, and reviews.

The implementation includes three stored procedures: `sp_InsertGame`, `sp_UpdateGame`, and `sp_DeleteGame`. These procedures centralize data modification logic and prevent invalid game data from being inserted, updated, or deleted directly.

5.1.1 Selected Table and Operations

The selected table for this part is GAME. This table stores the main product information of each game, including title, base price, release date, system requirements, and developer ID.

Table 5.1: Stored procedures implemented for the GAME table

Procedure	Purpose
<code>sp_InsertGame</code>	Inserts a new game into the GAME table after validating input values and developer existence.
<code>sp_UpdateGame</code>	Updates an existing game after checking that the game exists and that the new values are valid.
<code>sp_DeleteGame</code>	Deletes a game only if the game exists and is not already owned by players in their libraries.

5.1.2 Procedure-level Validation

The semantic constraints and business rules of the system were already discussed in Section 1.3. Therefore, this subsection only explains how the stored procedures apply relevant validation before modifying the GAME table.

The procedures validate required fields, data length, non-negative base price, reasonable release date, developer existence, duplicate game titles, and whether a game can be safely deleted. In particular, the delete procedure does not allow deleting a game that already exists in players' libraries, because this would break ownership records and affect historical user data.

When invalid input is detected, the procedures use `SIGNAL SQLSTATE '45000'` with specific error messages instead of generic error notifications.

5.1.3 Full SQL Implementation

The following SQL script shows the complete implementation of the three stored procedures (sp_InsertGame, sp_UpdateGame, and sp_DeleteGame) used by the group for the GAME table.

Listing 5.1: Full SQL script for the insert, update, and delete stored procedures on the GAME table

```
1 DELIMITER //
2
3 -- =====
4 -- 1. INSERT PROCEDURE
5 -- =====
6 CREATE PROCEDURE sp_InsertGame(
7     IN p_title VARCHAR(100),
8     IN p_base_price DECIMAL(10,2),
9     IN p_release_date DATE,
10    IN p_graphics VARCHAR(50),
11    IN p_os VARCHAR(50),
12    IN p_processor VARCHAR(50),
13    IN p_memory VARCHAR(50),
14    IN p_developer_id INT
15)
16 BEGIN
17     IF p_title IS NULL OR TRIM(p_title) = '' THEN
18         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Game
19             title cannot be empty.';
20     END IF;
21     IF CHAR_LENGTH(p_title) > 100 THEN
22         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Title
23             exceeds 100 characters.';
24     END IF;
25     IF CHAR_LENGTH(p_graphics) > 50 THEN
26         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
27             requirement [Graphics] fields cannot exceed 50
28             characters.';
29     END IF;
30     IF CHAR_LENGTH(p_os) > 50 THEN
31         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
```



```
requirement fields [OS] cannot exceed 50 characters.';
28 END IF;
29 IF CHAR_LENGTH(p_processor) > 50 THEN
30     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
        requirement fields [Processor] cannot exceed 50
        characters.';
31 END IF;
32 IF CHAR_LENGTH(p_memory) > 50 THEN
33     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
        requirement fields [Memory] cannot exceed 50 characters.
        ';
34 END IF;
35 IF p_base_price IS NULL THEN
36     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Base
        price cannot be empty.';
37 END IF;
38 IF p_base_price < 0 THEN
39     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Base
        price cannot be a negative value.';
40 END IF;
41 IF p_release_date > DATE_ADD(CURDATE(), INTERVAL 100 YEAR) THEN
42     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Release
        date is too far in the future.';
43 END IF;
44 IF p_release_date < '1950-01-01' THEN
45     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Release
        date is too far in the past.';
46 END IF;
47 IF p_developer_id IS NULL THEN
48     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error:
        Developer ID cannot be empty.';
49 END IF;
50 IF NOT EXISTS (SELECT 1 FROM `DEVELOPER` WHERE user_id =
        p_developer_id) THEN
51     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error:
        Developer ID does not exist.';
52 END IF;
53 IF EXISTS (SELECT 1 FROM `GAME` WHERE title = p_title) THEN
```

```
54     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: A game
      with this title already exists.';
55 END IF;
56 INSERT INTO `GAME` (title, base_price, release_date, graphics,
      os, processor, memory, developer_id)
57 VALUES (TRIM(p_title), p_base_price, p_release_date, p_graphics
      , p_os, p_processor, p_memory, p_developer_id);
58
59 SELECT 'Success: Game added successfully.' AS Result;
60 END //
61 -- =====
62 -- 2. UPDATE PROCEDURE
63 -- =====
64 CREATE PROCEDURE sp_UpdateGame(
65     IN p_game_id INT,
66     IN p_title VARCHAR(100),
67     IN p_base_price DECIMAL(10,2),
68     IN p_release_date DATE,
69     IN p_graphics VARCHAR(50),
70     IN p_os VARCHAR(50),
71     IN p_processor VARCHAR(50),
72     IN p_memory VARCHAR(50)
73 )
74 BEGIN
75     IF p_game_id IS NULL THEN
76         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Game ID
      cannot be empty.';
77     END IF;
78     IF NOT EXISTS (SELECT 1 FROM `GAME` WHERE game_id = p_game_id)
      THEN
79         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Cannot
      update. Game ID not found.';
80     END IF;
81     IF p_title IS NULL OR TRIM(p_title) = '' THEN
82         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Game
      title cannot be empty.';
83     END IF;
84     IF CHAR_LENGTH(p_title) > 100 THEN
```



```
85     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Title
        exceeds 100 characters.';
86 END IF;
87 IF CHAR_LENGTH(p_graphics) > 50 THEN
88     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
        requirement [Graphics] fields cannot exceed 50
        characters.';
89 END IF;
90 IF CHAR_LENGTH(p_os) > 50 THEN
91     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
        requirement fields [OS] cannot exceed 50 characters.';
92 END IF;
93 IF CHAR_LENGTH(p_processor) > 50 THEN
94     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
        requirement fields [Processor] cannot exceed 50
        characters.';
95 END IF;
96 IF CHAR_LENGTH(p_memory) > 50 THEN
97     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: System
        requirement fields [Memory] cannot exceed 50 characters.
        ';
98 END IF;
99 IF p_base_price IS NULL THEN
100     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Base
        price cannot be empty.';
101 END IF;
102 IF p_base_price < 0 THEN
103     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Base
        price cannot be a negative value.';
104 END IF;
105 IF p_release_date > DATE_ADD(CURDATE(), INTERVAL 100 YEAR) THEN
106     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Release
        date is too far in the future.';
107 END IF;
108 IF p_release_date < '1950-01-01' THEN
109     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Release
        date is too far in the past.';
110 END IF;
```



```
111 IF EXISTS (SELECT 1 FROM `GAME` WHERE title = p_title AND
112           game_id != p_game_id) THEN
113     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: A game
114           with this title already exists.';
115 END IF;
116 UPDATE `GAME`
117 SET title = TRIM(p_title),
118     base_price = p_base_price,
119     release_date = p_release_date,
120     graphics = p_graphics,
121     os = p_os,
122     processor = p_processor,
123     memory = p_memory
124 WHERE game_id = p_game_id;
125 SELECT 'Success: Game updated successfully.' AS Result;
126 END //
127 -- =====
128 -- 3. DELETE PROCEDURE
129 -- =====
130 CREATE PROCEDURE sp_DeleteGame(
131   IN p_game_id INT
132 )
133 BEGIN
134   IF p_game_id IS NULL THEN
135     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Game ID
136           cannot be empty.';
137   END IF;
138   IF NOT EXISTS (SELECT 1 FROM `GAME` WHERE game_id = p_game_id)
139     THEN
140     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Deletion
141           failed. Game ID does not exist.';
142   END IF;
143   IF EXISTS (SELECT 1 FROM `OWNS_LIBRARY` WHERE game_id =
144             p_game_id) THEN
145     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Cannot
146           delete game. It is already owned by players in their
147           library.';
148   END IF;
```

```
141 DELETE FROM `GAME` WHERE game_id = p_game_id;
142 SELECT 'Success: Game deleted successfully.' AS Result;
143 END //
144 DELIMITER ;
```

5.1.4 Test Cases

After creating the stored procedures, the following SQL statements were used to test the insert, update, and delete operations on the GAME table. A temporary game record was inserted first, then updated, and finally deleted. The variable @test_game_id was used to store the ID of the newly inserted game so that the update and delete procedures could be tested on the same record.

Listing 5.2: Test cases for GAME insert, update, and delete procedures

```
1 USE steam_store;
2
3 -- Test 1: Insert a new game
4 CALL sp_InsertGame(
5     'Test Game For Procedure',
6     250000.00,
7     '2026-05-01',
8     'GTX 1050',
9     'Windows 10',
10    'Intel i5',
11    '8GB RAM',
12    6
13 );
14
15 -- Check inserted game
16 SELECT *
17 FROM `GAME`
18 WHERE title = 'Test Game For Procedure';
19
20 -- Save inserted game ID
21 SET @test_game_id = (
22     SELECT game_id
23     FROM `GAME`
24     WHERE title = 'Test Game For Procedure'
```

```
25     LIMIT 1
26 );
27
28 -- Test 2: Update the inserted game
29 CALL sp_UpdateGame(
30     @test_game_id,
31     'Updated Test Game For Procedure',
32     300000.00,
33     '2026-05-02',
34     'GTX 1060',
35     'Windows 11',
36     'Intel i5',
37     '8GB RAM'
38 );
39
40 -- Check updated game
41 SELECT *
42 FROM `GAME`
43 WHERE game_id = @test_game_id;
44
45 -- Test 3: Delete the inserted game
46 CALL sp_DeleteGame(@test_game_id);
47
48 -- Check deleted game
49 SELECT *
50 FROM `GAME`
51 WHERE game_id = @test_game_id;
```

5.1.5 Execution Result

The test results show that the three procedures worked correctly. First, `sp_InsertGame` successfully inserted a new game into the `GAME` table. Then, `sp_UpdateGame` updated the inserted game using the saved game ID. Finally, `sp_DeleteGame` removed the test game from the table.

```

14 • SELECT *
15 FROM `GAME`
16 WHERE title = 'Test Game For Procedure';

```

game_id	title	base_price	release_date	graphics	os	processor	memory	developer_id
12	Test Game For Procedure	250000.00	2026-05-01	GTX 1050	Windows 10	Intel i5	8GB RAM	6

Figure 5.1: Result of inserting a new game using sp_InsertGame

```

19 • CALL sp_UpdateGame(
20     @test_game_id,
21     'Updated Test Game For Procedure',
22     300000.00,
23     '2026-05-02',
24     'GTX 1060',
25     'Windows 11',
26     'Intel i5',
27     '8GB RAM'
28 );
29
30 • SELECT *
31 FROM `GAME`
32 WHERE game_id = @test_game_id;

```

game_id	title	base_price	release_date	graphics	os	processor	memory	developer_id
13	Updated Test Game For Procedure	300000.00	2026-05-02	GTX 1060	Windows 11	Intel i5	8GB RAM	6

Figure 5.2: Result of updating the inserted game using sp_UpdateGame

```

34 -- Test 3: Delete the inserted game
35 • CALL sp_DeleteGame(@test_game_id);
36
37 -- Check deleted game
38 • SELECT *
39 FROM `GAME`
40 WHERE game_id = @test_game_id;

```

game_id	title	base_price	release_date	graphics	os	processor	memory	developer_id

Figure 5.3: Result of deleting the test game using sp_DeleteGame

5.2 Triggers

This section presents the triggers implemented in the Steam Store Management database. According to the assignment requirement, two categories of triggers must be provided: one trigger that enforces a **business constraint** which cannot be expressed directly inside table creation statements, and one or more triggers that compute the value of a **derived attribute**.

The first part introduces the constraint trigger `trg_review_only_owned_game`, which enforces the semantic rule that a player may submit a review for a game only if that game already exists in the player's library. The second part introduces a chain of triggers that maintain two derived attributes: `effective_price` in the `GAME` table and `price_at_purchase` in the `CONTAINS` table.

5.2.1 Business Constraint Trigger

A semantic constraint identified in Section 1.3 states that *a player may write a review for a game only if that game already exists in the player's account or library*. This rule cannot be enforced directly with a `CHECK` constraint inside the `REVIEW` table, because verifying it requires looking up rows in another table (`OWNS_LIBRARY`) at the moment a review is inserted or updated. Therefore, a trigger is the appropriate mechanism to enforce this business rule.

DML operations that may violate the constraint. The constraint can be violated by two DML operations on the `REVIEW` table: an `INSERT` that adds a new review for a game the player does not own, and an `UPDATE` that changes the `game_id` or `user_id` of an existing review so that the new combination is no longer present in `OWNS_LIBRARY`. Two triggers are therefore implemented, one for each operation.

Trigger implementation. The trigger queries the `OWNS_LIBRARY` table to check whether a row with the same `user_id` and `game_id` as the new review exists. If no such row is found, the trigger raises a `SIGNAL SQLSTATE '45000'` with a meaningful error message instead of allowing the invalid review to be inserted or updated.

Listing 5.3: Trigger enforcing the business rule that reviews are allowed only for owned games

```
1 DELIMITER //
```

```
2
```

```
3 CREATE TRIGGER trg_review_only_owned_game_insert
```

```
4 BEFORE INSERT ON REVIEW
```

```
5 FOR EACH ROW
```

```
6 BEGIN
```

```
7     DECLARE v_owned INT DEFAULT 0;
```

```
8
```



```
9      SELECT COUNT(*) INTO v_owned
10     FROM OWNS_LIBRARY
11     WHERE player_id = NEW.player_id
12           AND game_id = NEW.game_id;
13
14     IF v_owned = 0 THEN
15         SIGNAL SQLSTATE '45000'
16         SET MESSAGE_TEXT =
17             'Constraint Violation: A player may only review a game
18              they own in their library.';
19     END IF;
20
21 END //
22
23 CREATE TRIGGER trg_review_only_owned_game_update
24 BEFORE UPDATE ON REVIEW
25 FOR EACH ROW
26 BEGIN
27     DECLARE v_owned INT DEFAULT 0;
28
29     IF NEW.player_id <> OLD.player_id OR NEW.game_id <> OLD.game_id
30     THEN
31         SELECT COUNT(*) INTO v_owned
32         FROM OWNS_LIBRARY
33         WHERE player_id = NEW.player_id
34               AND game_id = NEW.game_id;
35
36         IF v_owned = 0 THEN
37             SIGNAL SQLSTATE '45000'
38             SET MESSAGE_TEXT =
39                 'Constraint Violation: Updated review must still
40                  refer to a game owned by the player.';
41         END IF;
42     END IF;
43 END //
44
45 DELIMITER ;
```

Table 5.2: Test cases for the review-only-on-owned-game constraint trigger

No.	Test scenario	Expected outcome
1	A player attempts to review a game that exists in their OWNS_LIBRARY.	Review is inserted successfully.
2	A player attempts to review a game that does <i>not</i> exist in their OWNS_LIBRARY.	Trigger raises error 45000 with the message “A player may only review a game they own in their library.”
3	An existing review is updated to point to a different game that the player does <i>not</i> own.	Update is rejected by trg_review_only_owned_game_update.

Listing 5.4: Sample SQL statements to demonstrate the constraint trigger
Test cases for the constraint trigger.

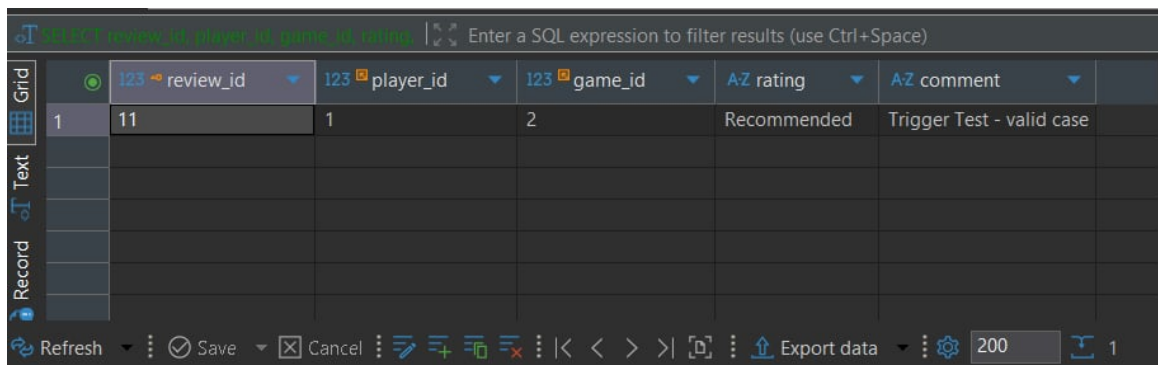
```
1 USE SteamDB;
2
3 -- =====
4 -- Test 1 (VALID): player_id = 1 owns game_id = 2 in OWNS_LIBRARY,
5 -- and no review exists yet for (1, 2). The trigger allows the
6 -- insert.
7 -- =====
8 INSERT INTO `REVIEW` (rating, comment, review_date, player_id,
9 game_id)
10 VALUES ('Recommended', 'Trigger Test - valid case', CURDATE(), 1,
11 2);
12
13 SELECT review_id, player_id, game_id, rating, comment
14 FROM `REVIEW`
15 WHERE player_id = 1 AND game_id = 2;
16 -- Expected: 1 row inserted successfully.
17
18 -- =====
19 -- Test 2 (INVALID INSERT): player_id = 1 does NOT own game_id = 7
20 -- (player 1 only owns games 2 and 3 in OWNS_LIBRARY).
21 -- The trigger must reject the insert.
22 -- =====
```



```
21 INSERT INTO `REVIEW` (rating, comment, review_date, player_id,
    game_id)
22 VALUES ('Recommended', 'Trigger Test - should fail', CURDATE(), 1,
    7);
23 -- Expected:
24 -- ERROR 1644 (45000): Constraint Violation: A player may only
    review
25 -- a game they own in their library.
26
27
28 -- =====
29 -- Test 3 (INVALID UPDATE): change the review just inserted in Test
    1
30 -- (player 1, game 2) so that it now points to game 7 - which
    player 1
31 -- does NOT own. The UPDATE trigger must reject the change.
32 -- =====
33 UPDATE `REVIEW`
34 SET game_id = 7
35 WHERE player_id = 1 AND game_id = 2;
36 -- Expected:
37 -- ERROR 1644 (45000): Constraint Violation: Updated review must
    still refer
38 -- to a game owned by the player.
39
40
41 -- =====
42 -- Cleanup (optional): remove the test review inserted in Test 1
    so the database returns to its original state.
43 -- =====
44
45 DELETE FROM `REVIEW`
46 WHERE player_id = 1 AND game_id = 2 AND comment = 'Trigger Test -
    valid case';
```

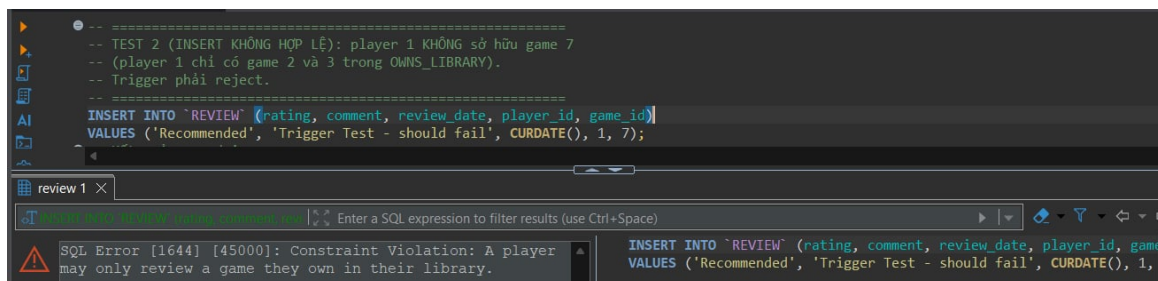
Execution result. The screenshots below show the execution of the three test cases inside the MySQL client. Test 1 (Figure 5.4) inserts a new review for player 1 and game 2 successfully because player 1 owns game 2 in OWNS_LIBRARY; the subsequent SELECT confirms

that the row was created with `review_id = 11`. Test 2 (Figure 5.5) attempts to insert a review for player 1 on game 7, which is not owned by player 1; the trigger rejects the operation with SQL Error 1644 [45000]: Constraint Violation: A player may only review a game they own in their library. Test 3 (Figure 5.6) attempts to update the review just inserted in Test 1 so that its `game_id` becomes 7; the update trigger rejects the change with SQL Error 1644 [45000]: Constraint Violation: Updated review must still refer to a game owned by the player.



	review_id	player_id	game_id	rating	comment
1	11	1	2	Recommended	Trigger Test - valid case

Figure 5.4: Test 1 result — valid insert succeeds (player 1 owns game 2)

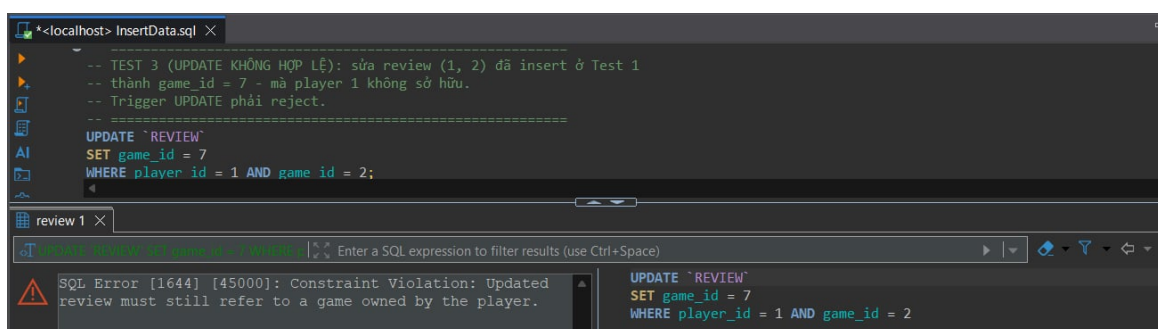


```
-- TEST 2 (INSERT KHÔNG HỢP LỆ): player 1 KHÔNG sở hữu game 7
-- (player 1 chỉ có game 2 và 3 trong OWNS_LIBRARY).
-- Trigger phải reject.

INSERT INTO 'REVIEW' (rating, comment, review_date, player_id, game_id)
VALUES ('Recommended', 'Trigger Test - should fail', CURDATE(), 1, 7);
```

SQL Error [1644] [45000]: Constraint Violation: A player may only review a game they own in their library.

Figure 5.5: Test 2 result — invalid insert rejected by
`trg_review_only_owned_game_insert`



```
-- TEST 3 (UPDATE KHÔNG HỢP LỆ): sửa review (1, 2) đã insert ở Test 1
-- thành game_id = 7 - mà player 1 không sở hữu.
-- Trigger UPDATE phải reject.

UPDATE 'REVIEW'
SET game_id = 7
WHERE player_id = 1 AND game_id = 2;
```

SQL Error [1644] [45000]: Constraint Violation: Updated review must still refer to a game owned by the player.

Figure 5.6: Test 3 result — invalid update rejected by
`trg_review_only_owned_game_update`

5.2.2 Chosen Derived Attribute

The chosen derived attribute is `price_at_purchase` from the `CONTAINS` relationship table. This attribute stores the price of a game at the moment of purchase. It is important because the price of a game may change later due to base price updates or promotions, but historical transaction records must preserve the exact price paid at the time of purchase.

The value of `price_at_purchase` is derived from the current `effective_price` of the corresponding game. If `effective_price` is not available, the system falls back to the game's `base_price`.

5.2.3 DML Operations Affecting the Derived Attributes

Since `price_at_purchase` depends on `effective_price`, the triggers must handle both attributes. Table 5.3 shows the DML operations that affect `effective_price`.

Table 5.3: DML operations affecting `effective_price`

DML Operation	Table	Reason
Insert	GAME	A newly inserted game has no promotion, so its <code>effective_price</code> is initialized using its <code>base_price</code> .
Update	GAME	When <code>base_price</code> changes, the <code>effective_price</code> must be recalculated.
Insert	GAME_PROMOTION	When a promotion is assigned to a game, the <code>effective_price</code> may change.
Delete	PROMOTION	When a promotion is removed, affected games must recalculate their <code>effective_price</code> .

Table 5.4 shows the DML operation that affects `price_at_purchase`.

Table 5.4: DML operation affecting `price_at_purchase`

DML Operation	Table	Reason
Insert	CONTAINS	When a game is inserted into a transaction, the current <code>effective_price</code> must be copied into <code>price_at_purchase</code> .

5.2.4 Full SQL Implementation

The following SQL script shows the triggers used to maintain `effective_price` and `price_at_purchase`.

Listing 5.5: Triggers for maintaining effective price and price at purchase

```
1 DELIMITER //
2
3 CREATE TRIGGER trg_contains_set_price
4 BEFORE INSERT ON CONTAINS
5 FOR EACH ROW
6 BEGIN
7     DECLARE v_effective_price DECIMAL(10,2);
8
9     SELECT COALESCE(effective_price, base_price) INTO
10        v_effective_price
11 FROM GAME
12 WHERE game_id = NEW.game_id;
13
14 SET NEW.price_at_purchase = v_effective_price;
15
16 END //
17
18 DELIMITER ;
19
20 DELIMITER //
21
22 CREATE TRIGGER trg_game_insert_price
23 BEFORE INSERT ON GAME
24 FOR EACH ROW
25 BEGIN
26     SET NEW.effective_price = NEW.base_price;
27
28 END //
29
30 DELIMITER ;
31
32 DELIMITER //
33
34 CREATE TRIGGER trg_update_game_price
35 BEFORE UPDATE ON GAME
```



```
33 FOR EACH ROW
34 BEGIN
35     DECLARE v_discount DECIMAL(5,2);
36
37     IF OLD.base_price <> NEW.base_price THEN
38         SELECT COALESCE(MAX(p.discount_percentage), 0)
39         INTO v_discount
40         FROM PROMOTION p
41         JOIN GAME_PROMOTION gp ON p.promotion_id = gp.promotion_id
42         WHERE gp.game_id = NEW.game_id
43             AND CURDATE() BETWEEN p.start_date AND p.end_date;
44
45         SET NEW.effective_price = ROUND(NEW.base_price * (1 -
46             v_discount / 100), 2);
47     END IF;
48
49 DELIMITER ;
50
51 DELIMITER //
52
53 CREATE TRIGGER trg_gamepromotion_insert
54 AFTER INSERT ON GAME_PROMOTION
55 FOR EACH ROW
56 BEGIN
57     DECLARE v_discount DECIMAL(5,2);
58
59     SELECT COALESCE(MAX(p.discount_percentage), 0)
60     INTO v_discount
61     FROM PROMOTION p
62     JOIN GAME_PROMOTION gp
63         ON p.promotion_id = gp.promotion_id
64     WHERE gp.game_id = NEW.game_id
65         AND CURDATE() BETWEEN p.start_date AND p.end_date;
66
67     UPDATE GAME
68     SET effective_price = ROUND(base_price * (1 - v_discount / 100)
69         , 2)
```

```
69     WHERE game_id = NEW.game_id;
70 END //
71
72 DELIMITER ;
73
74 DELIMITER //
75
76 CREATE TRIGGER trg_promotion_delete
77 BEFORE DELETE ON PROMOTION
78 FOR EACH ROW
79 BEGIN
80     DECLARE done INT DEFAULT 0;
81     DECLARE v_game_id INT;
82     DECLARE v_base_price DECIMAL(10,2);
83     DECLARE v_discount DECIMAL(5,2);
84
85     DECLARE game_cursor CURSOR FOR
86         SELECT g.game_id, g.base_price
87         FROM GAME g
88         JOIN GAME_PROMOTION gp ON g.game_id = gp.game_id
89         WHERE gp.promotion_id = OLD.promotion_id;
90
91     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
92
93     OPEN game_cursor;
94
95     read_loop: LOOP
96         FETCH game_cursor INTO v_game_id, v_base_price;
97         IF done THEN
98             LEAVE read_loop;
99         END IF;
100
101         SELECT COALESCE(MAX(p.discount_percentage), 0)
102         INTO v_discount
103         FROM PROMOTION p
104         JOIN GAME_PROMOTION gp2 ON p.promotion_id = gp2.
105             promotion_id
106         WHERE gp2.game_id = v_game_id
```

```
106         AND p.promotion_id <> OLD.promotion_id
107         AND CURDATE() BETWEEN p.start_date AND p.end_date;
108
109     UPDATE GAME
110     SET effective_price = ROUND(v_base_price * (1 - v_discount
111                               / 100), 2)
112     WHERE game_id = v_game_id;
113
114 END LOOP;
115
116 CLOSE game_cursor;
117
118 END //
```

```
118 DELIMITER ;
```

5.2.5 Trigger Explanation

The trigger `trg_game_insert_price` is executed before a new game is inserted into the `GAME` table. Since a newly inserted game has no promotion applied yet, its `effective_price` is initialized with the same value as its `base_price`.

The trigger `trg_update_game_price` is executed before a row in the `GAME` table is updated. When the `base_price` changes, the trigger retrieves the highest active discount percentage among all valid promotions associated with that game. The `effective_price` is then recalculated using the updated base price and the best available discount.

The trigger `trg_gamepromotion_insert` is executed after a new record is inserted into the `GAME_PROMOTION` table. When a promotion is assigned to a game, this trigger updates the game's `effective_price`. If multiple active promotions are available, the highest discount percentage is selected using the `MAX` function.

The trigger `trg_promotion_delete` is executed before a promotion is deleted from the `PROMOTION` table. Since one promotion may affect multiple games, a cursor is used to iterate through all affected games. For each game, the trigger recalculates the `effective_price` using the highest remaining active promotion. If no valid promotion remains, the discount defaults to 0.

Finally, the trigger `trg_contains_set_price` is executed before a new record is inserted into the `CONTAINS` table. It copies the current `effective_price` of the game into `price_at_purchase`. This ensures that each transaction stores the exact game price at the moment of purchase and preserves historical accuracy even if the game price changes later.

5.2.6 Test Cases

The following SQL statements were used to test the trigger behavior.

Listing 5.6: Test cases for price-related triggers

```
1 USE steam_store;
2
3 -- Test 1: Check that effective_price is initialized
4 -- when a new game is inserted
5 INSERT INTO `GAME` (
6     title, base_price, release_date, graphics, os, processor,
7     memory, developer_id
8 )
9 VALUES (
10     'Trigger Test Game',
11     1000000.00,
12     '2026-06-01',
13     'GTX 1050',
14     'Windows 10',
15     'Intel i5',
16     '8GB RAM',
17     6
18 );
19 SET @test_game_id = LAST_INSERT_ID();
20
21 SELECT
22     game_id,
23     title,
24     base_price,
25     effective_price
26 FROM `GAME`
27 WHERE game_id = @test_game_id;
28
29 -- Test 2: Apply an active promotion to the game
30 INSERT INTO `PROMOTION` (
31     name, discount_percentage, start_date, end_date
32 )
33 VALUES (
```



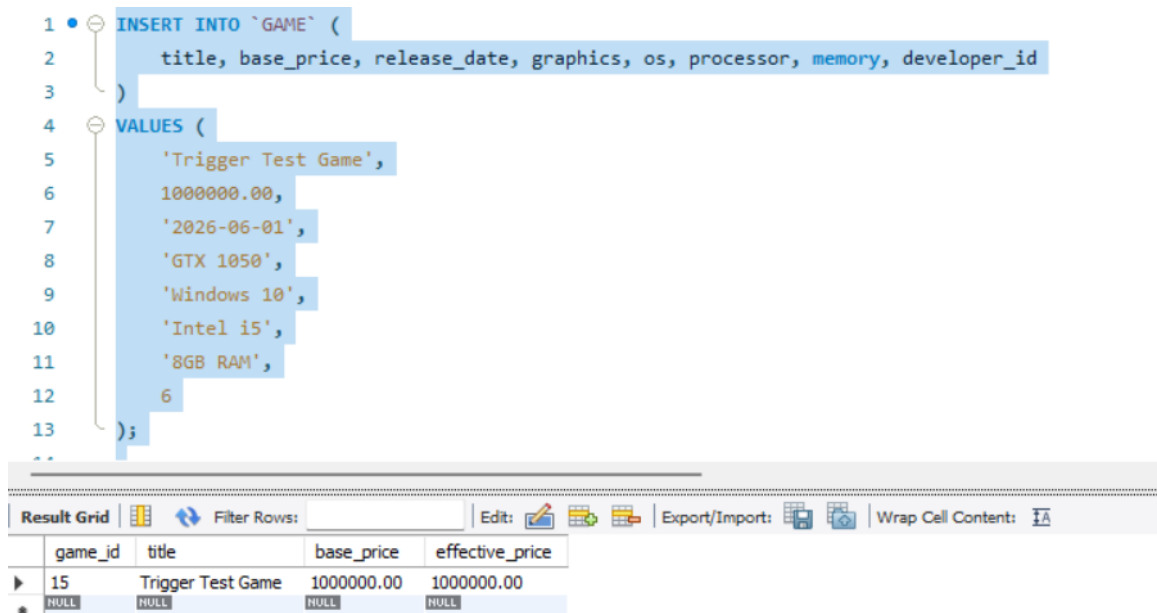
```
34     'Trigger Test Promotion',
35     20.00,
36     CURDATE(),
37     DATE_ADD(CURDATE(), INTERVAL 10 DAY)
38 );
39
40 SET @test_promotion_id = LAST_INSERT_ID();
41
42 INSERT INTO `GAME_PROMOTION` (game_id, promotion_id)
43 VALUES (@test_game_id, @test_promotion_id);
44
45 SELECT
46     game_id,
47     title,
48     base_price,
49     effective_price
50 FROM `GAME`
51 WHERE game_id = @test_game_id;
52
53 -- Test 3: Insert the game into a transaction
54 INSERT INTO `TRANSACTION` (
55     date, payment_method, total_amount, player_id
56 )
57 VALUES (
58     NOW(),
59     'Credit Card',
60     800000.00,
61     1
62 );
63
64 SET @test_transaction_id = LAST_INSERT_ID();
65
66 INSERT INTO `CONTAINS` (
67     transaction_id, game_id, price_at_purchase
68 )
69 VALUES (
70     @test_transaction_id,
71     @test_game_id,
```



```
72 0
73 );
74
75 SELECT
76     transaction_id,
77     game_id,
78     price_at_purchase
79 FROM `CONTAINS`
80 WHERE transaction_id = @test_transaction_id
81 AND game_id = @test_game_id;
```

5.2.7 Execution Result

The test results show that the triggers correctly maintain both derived attributes. First, when a new game is inserted into the GAME table, its effective_price is initialized from the base_price. Then, when an active promotion with a 20% discount is assigned to the game, the effective_price is automatically recalculated from 1,000,000.00 to 800,000.00. Finally, when the game is inserted into a transaction through the CONTAINS table, the current effective_price is copied into price_at_purchase.



The screenshot displays a database IDE interface. The top section shows an SQL query being executed:

```
1 INSERT INTO `GAME` (  
2     title, base_price, release_date, graphics, os, processor, memory, developer_id  
3 )  
4 VALUES (  
5     'Trigger Test Game',  
6     1000000.00,  
7     '2026-06-01',  
8     'GTX 1050',  
9     'Windows 10',  
10    'Intel i5',  
11    '8GB RAM',  
12    6  
13 );
```

Below the query editor, the 'Result Grid' is visible, showing the data inserted into the GAME table:

game_id	title	base_price	effective_price
15	Trigger Test Game	1000000.00	1000000.00
NULL	NULL	NULL	NULL

Figure 5.7: Trigger result after applying an active promotion to update effective_price

Result Grid			
	transaction_id	game_id	price_at_purchase
▶	17	15	800000.00
•	NULL	NULL	NULL

Figure 5.8: Trigger result for storing price_at_purchase in the CONTAINS table

5.3 Stored Procedures for Data Retrieval

This section presents stored procedures used to retrieve meaningful information from the database. Unlike the procedures in Section 2.1, these procedures do not modify data. Instead, they support searching, filtering, joining related tables, aggregation, grouping, and sorting results for the application interface.

The first procedure retrieves games by genre and maximum price. The second procedure retrieves developers whose revenue exceeds a given threshold within a selected date range. Together, these procedures demonstrate how stored procedures can support realistic data retrieval operations in a Steam-like digital distribution platform.

5.3.1 Procedure 1: Get Games by Genre and Price

The procedure `GetGamesByGenreAndPrice` retrieves games that belong to a selected genre and have a base price not greater than a given maximum price. This procedure is similar to a filtering feature in a digital game store, where users can browse games by category and price range.

The procedure joins the `GAME`, `GAME_GENRE`, and `DEVELOPER` tables. The parameter `p_GenreName` is used to filter games by genre, while `p_MaxPrice` limits the result to games whose base price does not exceed the selected value. The result is ordered by base price in descending order and then by game title in ascending order.

Listing 5.7: Procedure for retrieving games by genre and maximum price (column aliases shown in English; the actual SQL file in `SQLData/Procedure1.sql` uses Vietnamese aliases for end-user display)

```

1 DELIMITER $$
2
3 CREATE PROCEDURE GetGamesByGenreAndPrice (
4     IN p_GenreName VARCHAR(50),
5     IN p_MaxPrice DECIMAL(10,2)
6 )
7 BEGIN

```

```
8 SELECT
9     g.game_id,
10    g.title          AS `Game Title`,
11    g.base_price     AS `Base Price`,
12    g.release_date   AS `Release Date`,
13    d.legal_name     AS `Developer`
14 FROM `GAME` g
15 JOIN `GAME_GENRE` gg ON g.game_id = gg.game_id
16 JOIN `DEVELOPER` d ON g.developer_id = d.user_id
17 WHERE gg.genre_name = p_GenreName
18        AND g.base_price <= p_MaxPrice
19 ORDER BY g.base_price DESC, g.title ASC;
20 END$$
```

5.3.2 Procedure 2: Get Developer Revenue

The procedure `GetDeveloperRevenue` retrieves developers whose total revenue is greater than or equal to a given threshold within a selected date range. This procedure supports the developer-side view of the platform, where developers or administrators may want to analyze sales performance.

The procedure calculates revenue from sold games by joining the `DEVELOPER`, `GAME`, `CONTAINS`, and `TRANSACTION` tables. It uses two aggregate functions, `COUNT(c.game_id)` for the total number of sold copies and `SUM(c.price_at_purchase)` for the total revenue, groups the result by developer, filters developers using the `HAVING` clause (`SUM(c.price_at_purchase) >= p_MinRevenue`), and orders the output by total revenue in descending order.

Listing 5.8: Procedure for retrieving developer revenue (column aliases shown in English; the actual SQL file uses Vietnamese aliases such as Nhà Phát Triển, Tổng Số Lượt Bán, Tổng Doanh Thu)

```
1 CREATE PROCEDURE GetDeveloperRevenue (
2     IN p_StartDate  DATE,
3     IN p_EndDate    DATE,
4     IN p_MinRevenue DECIMAL(15,2)
5 )
6 BEGIN
7     SELECT
8         d.legal_name          AS `Developer`,
9         COUNT(c.game_id)      AS `Total Sales Count`,
```

```
10      SUM(c.price_at_purchase) AS `Total Revenue`
11 FROM `DEVELOPER` d
12 JOIN `GAME`      g ON d.user_id = g.developer_id
13 JOIN `CONTAINS`  c ON g.game_id = c.game_id
14 JOIN `TRANSACTION` t ON c.transaction_id = t.transaction_id
15 WHERE DATE(t.date) >= p_StartDate
16      AND DATE(t.date) <= p_EndDate
17 GROUP BY d.user_id, d.legal_name
18 HAVING SUM(c.price_at_purchase) >= p_MinRevenue
19 ORDER BY SUM(c.price_at_purchase) DESC;
20 END$$
21
22 DELIMITER ;
```

5.3.3 Test Cases

The following SQL statements were used to test the data retrieval procedures. The first test retrieves RPG games whose base price is under 1,000,000. The second test retrieves developers whose revenue is greater than 500,000 from 2023 to the end of 2024.

Listing 5.9: Test cases for data retrieval procedures

```
1 -- Test Procedure 1: Find RPG games priced under 1,000,000
2 CALL GetGamesByGenreAndPrice('RPG', 1000000);
3 -- Test Procedure 2: List Developers with revenue over 500,000
4 -- from 2023 through end of 2024
5 CALL GetDeveloperRevenue('2023-01-01', '2024-12-31', 500000);
```

5.3.4 Execution Result

The execution results show that the retrieval procedures return meaningful information from the database. The first procedure returns RPG games whose base price does not exceed 1,000,000. The second procedure returns developers whose total revenue is greater than 500,000 during the selected period.

```
31 • -- Test Procedure 1: Find RPG games priced under 1,000,000
32 CALL GetGamesByGenreAndPrice('RPG', 1000000);
```

game_id	Game Title	Base Price	Release Date	Developer
7	Red Dead Redemption 2	1000000.00	2019-11-05	Rockstar Games
3	Cyberpunk 2077	990000.00	2020-12-10	CD Projekt S.A.
10	Monster Hunter: World	600000.00	2018-08-09	Capcom Co., Ltd.
11	The Witcher 3: Wild Hunt	400000.00	2015-05-18	CD Projekt S.A.

Figure 5.9: Result of retrieving RPG games priced under 1,000,000

Nhà Phát Triển	Tổng Số Lượt Bán	Tổng Doanh Thu
Rockstar Games	5	4000000.00
Square Enix Co., Ltd.	2	3000000.00
Capcom Co., Ltd.	3	3000000.00
CD Projekt S.A.	3	1790000.00
Valve Corporation	1	1200000.00

Figure 5.10: Result of retrieving developers with revenue over 500,000

5.4 Stored Functions

This section presents two stored functions used to calculate business values from the database. Unlike stored procedures, these functions return a single calculated value and can be used directly inside SELECT statements.

The first function calculates the total spending of a player within a selected date range. The second function calculates the net revenue of a developer after subtracting the platform fee. Both functions include input validation and use cursors to process related records step by step.

5.4.1 Function 1: Calculate Player Spending

The function `fn_CalculatePlayerSpending` calculates the total amount spent by a player within a selected date range. It receives a player ID, a starting date, and an ending date as input parameters.

The function first validates the input values. It checks whether the player ID is empty, whether the player exists, whether the date range is empty, and whether the starting date is later than the ending date. After validation, it uses a cursor to iterate through all transactions of the selected player within the given period and accumulates the transaction amounts into a total spending value.

This function is useful for player-side analysis, such as showing how much a player has spent on the platform during a specific period.

5.4.2 Function 2: Calculate Developer Net Revenue

The function `fn_GetDeveloperNetRevenue` calculates the net revenue of a developer within a selected date range after subtracting a platform fee percentage. It receives a developer ID, a starting date, an ending date, and a platform fee percentage as input parameters.

This function is more advanced because it uses a cursor to iterate through all games published by the selected developer. For each game, it calculates revenue from base game sales using the `CONTAINS` table and revenue from DLC sales using the `BUY_DLC` and `DLC` tables. The game revenue and DLC revenue are accumulated into gross revenue. Finally, the platform fee is subtracted to return the developer's net revenue.

The function also validates important input conditions, including developer existence, valid date range, and valid platform fee percentage.

5.4.3 Full SQL Implementation

The following SQL script shows the full implementation of the two stored functions.

Listing 5.10: Stored functions for player spending and developer net revenue

```
1 USE steam_store;
2
3 DELIMITER //
4
5 DROP FUNCTION IF EXISTS fn_CalculatePlayerSpending //
6
7 CREATE FUNCTION fn_CalculatePlayerSpending(
8     p_player_id INT,
9     p_from_date DATE,
10    p_to_date DATE
11 )
12 RETURNS DECIMAL(14,2)
13 DETERMINISTIC
```

```
14 READS SQL DATA
15 BEGIN
16     DECLARE done INT DEFAULT 0;
17     DECLARE v_amount DECIMAL(14,2) DEFAULT 0;
18     DECLARE v_total_spending DECIMAL(14,2) DEFAULT 0;
19
20     DECLARE cur_player_transactions CURSOR FOR
21         SELECT IFNULL(t.total_amount, 0)
22         FROM `TRANSACTION` t
23         WHERE t.player_id = p_player_id
24             AND DATE(t.`date`) BETWEEN p_from_date AND p_to_date
25         ORDER BY t.`date`;
26
27     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
28
29     IF p_player_id IS NULL THEN
30         SIGNAL SQLSTATE '45000'
31             SET MESSAGE_TEXT = 'Player ID cannot be empty.';
32     END IF;
33
34     IF NOT EXISTS (SELECT 1 FROM `PLAYER` WHERE user_id =
35         p_player_id) THEN
36         SIGNAL SQLSTATE '45000'
37             SET MESSAGE_TEXT = 'Player ID does not exist.';
38     END IF;
39
40     IF p_from_date IS NULL OR p_to_date IS NULL THEN
41         SIGNAL SQLSTATE '45000'
42             SET MESSAGE_TEXT = 'Date range cannot be empty.';
43     END IF;
44
45     IF p_from_date > p_to_date THEN
46         SIGNAL SQLSTATE '45000'
47             SET MESSAGE_TEXT = 'From date cannot be later than to
48             date.';
49     END IF;
50
51     OPEN cur_player_transactions;
```



```
50
51 spending_loop: LOOP
52     FETCH cur_player_transactions INTO v_amount;
53
54     IF done = 1 THEN
55         LEAVE spending_loop;
56     END IF;
57
58     SET v_total_spending = v_total_spending + v_amount;
59 END LOOP;
60
61 CLOSE cur_player_transactions;
62
63 RETURN v_total_spending;
64 END //
```

```
65
66 DROP FUNCTION IF EXISTS fn_GetDeveloperNetRevenue //
```

```
67
68 CREATE FUNCTION fn_GetDeveloperNetRevenue (
69     p_developer_id INT,
70     p_from_date DATE,
71     p_to_date DATE,
72     p_platform_fee_percent DECIMAL(5,2)
73 )
74 RETURNS DECIMAL(14,2)
75 DETERMINISTIC
76 READS SQL DATA
77 BEGIN
78     DECLARE done INT DEFAULT 0;
79     DECLARE v_game_id INT;
80     DECLARE v_game_sales DECIMAL(14,2) DEFAULT 0;
81     DECLARE v_dlc_sales DECIMAL(14,2) DEFAULT 0;
82     DECLARE v_gross_revenue DECIMAL(14,2) DEFAULT 0;
83     DECLARE v_net_revenue DECIMAL(14,2) DEFAULT 0;
84
85     DECLARE cur_developer_games CURSOR FOR
86         SELECT g.game_id
87         FROM `GAME` g
```



```
88     WHERE g.developer_id = p_developer_id
89     ORDER BY g.game_id;
90
91 DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
92
93 IF p_developer_id IS NULL THEN
94     SIGNAL SQLSTATE '45000'
95     SET MESSAGE_TEXT = 'Developer ID cannot be empty.';
96 END IF;
97
98 IF NOT EXISTS (SELECT 1 FROM `DEVELOPER` WHERE user_id =
99     p_developer_id) THEN
100     SIGNAL SQLSTATE '45000'
101     SET MESSAGE_TEXT = 'Developer ID does not exist.';
102 END IF;
103
104 IF p_from_date IS NULL OR p_to_date IS NULL THEN
105     SIGNAL SQLSTATE '45000'
106     SET MESSAGE_TEXT = 'Date range cannot be empty.';
107 END IF;
108
109 IF p_from_date > p_to_date THEN
110     SIGNAL SQLSTATE '45000'
111     SET MESSAGE_TEXT = 'From date cannot be later than to
112     date.';
113 END IF;
114
115 IF p_platform_fee_percent IS NULL THEN
116     SIGNAL SQLSTATE '45000'
117     SET MESSAGE_TEXT = 'Platform fee cannot be empty.';
118 END IF;
119
120 IF p_platform_fee_percent < 0 OR p_platform_fee_percent > 100
121     THEN
122     SIGNAL SQLSTATE '45000'
123     SET MESSAGE_TEXT = 'Platform fee must be between 0 and
124     100.';
125 END IF;
```



```
122
123 OPEN cur_developer_games;
124
125 revenue_loop: LOOP
126     FETCH cur_developer_games INTO v_game_id;
127
128     IF done = 1 THEN
129         LEAVE revenue_loop;
130     END IF;
131
132     SELECT IFNULL(SUM(c.price_at_purchase), 0)
133     INTO v_game_sales
134     FROM `CONTAINS` c
135     JOIN `TRANSACTION` t
136         ON c.transaction_id = t.transaction_id
137     WHERE c.game_id = v_game_id
138         AND DATE(t.`date`) BETWEEN p_from_date AND p_to_date;
139
140     SELECT IFNULL(SUM(d.price), 0)
141     INTO v_dlc_sales
142     FROM `BUY_DLC` bd
143     JOIN `DLC` d
144         ON bd.game_id = d.game_id
145         AND bd.dlc_name = d.dlc_name
146     JOIN `TRANSACTION` t
147         ON bd.transaction_id = t.transaction_id
148     WHERE bd.game_id = v_game_id
149         AND DATE(t.`date`) BETWEEN p_from_date AND p_to_date;
150
151     SET v_gross_revenue = v_gross_revenue + v_game_sales +
152         v_dlc_sales;
153
154 CLOSE cur_developer_games;
155
156 SET v_net_revenue = v_gross_revenue * (1 -
157     p_platform_fee_percent / 100);
```

```
158     RETURN v_net_revenue;
159 END //
160
161 DELIMITER ;
```

5.4.4 Test Cases

The following SQL statements were used to test the stored functions. The first two test cases calculate total spending for two players. The last two test cases calculate net revenue for two developers after applying a 30% platform fee.

Listing 5.11: Test cases for stored functions

```
1 SELECT fn_CalculatePlayerSpending (
2     1,
3     '2023-01-01',
4     '2024-12-31'
5 ) AS player_1_total_spending;
6 SELECT fn_GetDeveloperNetRevenue (
7     16,
8     '2023-01-01',
9     '2024-12-31',
10    30
11 ) AS developer_16_net_revenue;
```

5.4.5 Execution Result

The execution results show that the functions return correct calculated values based on the sample data. The player spending function returns the total amount spent by each selected player during the given period. The developer net revenue function returns the revenue after subtracting the platform fee from gross game and DLC sales.

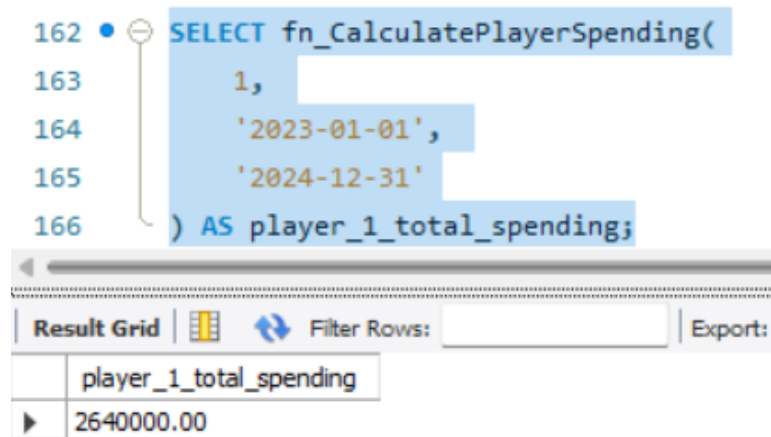


Figure 5.11: Result of calculating total player spending using stored functions

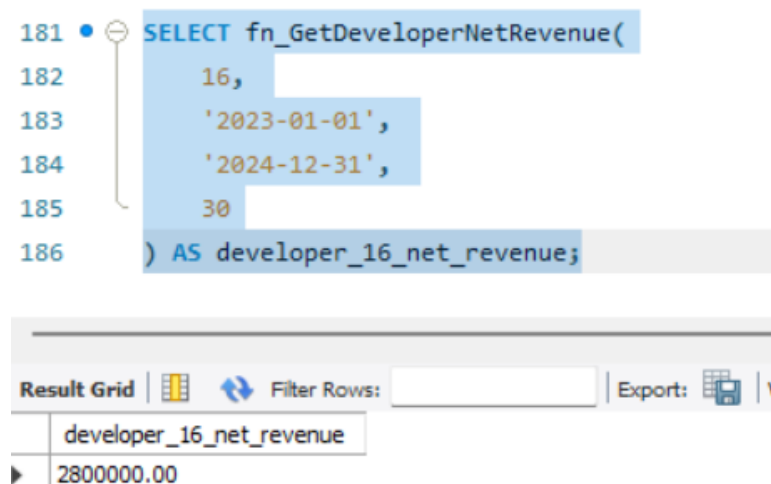


Figure 5.12: Result of calculating developer net revenue using stored functions

6 Application Implementation

6.1 CRUD Interface

This section presents the CRUD interface implemented for managing game records in the Steam Store Management application. The interface is designed for the GAME table, which stores the main information of each game, including title, base price, release date, system requirements, and developer ID.

The CRUD interface is implemented as a dedicated tab named *Add/Edit/Delete* in the application. It provides a form that allows users to enter game information and perform three basic operations: adding a new game, updating an existing game, and deleting a game. For update and delete operations, the user enters the target game_id. For insert operations, the user provides the required game information, including the developer ID.

6.1.1 Interface Design

The interface contains input fields for the main attributes of the GAME table, including game_id, title, base_price, release_date, graphics, os, processor, memory, and developer_id. Three buttons are provided for the user to execute CRUD actions: *Add Game*, *Update Game*, and *Delete Game*. The form is illustrated in Figure 6.1.

When an action is performed, the frontend sends an HTTP request to the backend server. The backend then calls the corresponding stored procedure in the MySQL database. This design allows the application to reuse the database logic implemented earlier in Section 2.1 instead of placing all validation and modification logic directly in the frontend.

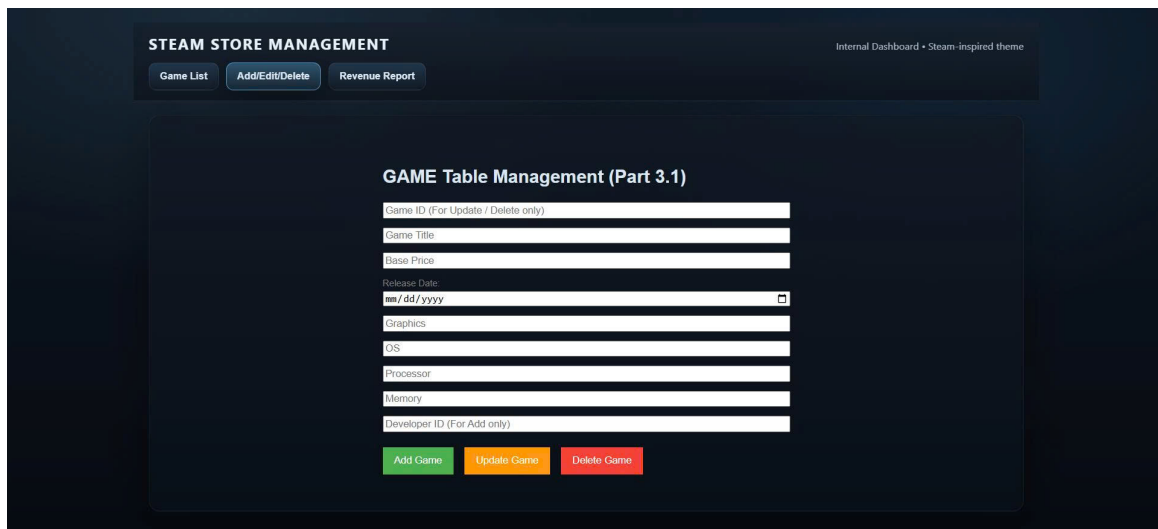


Figure 6.1: Add/Edit/Delete interface for the GAME table (Part 3.1)

Table 6.1: CRUD operations in the game management interface

Operation	Backend Endpoint	Stored Procedure
Add Game	POST /api/games	sp_InsertGame
Update Game	PUT /api/games/:id	sp_UpdateGame
Delete Game	DELETE /api/games/:id	sp_DeleteGame

6.1.2 Backend Connection

The backend is implemented using Node.js and Express. It receives requests from the React frontend and connects to the MySQL database. For each CRUD operation, the backend does not execute raw insert, update, or delete statements directly. Instead, it calls the stored procedures `sp_InsertGame`, `sp_UpdateGame`, and `sp_DeleteGame`.

This approach keeps the business rules and validation logic inside the database layer. If the input data violates a validation rule, the stored procedure returns an error message, and the frontend displays that message to the user.

After a successful CRUD operation, the user can switch to the *Game List* tab to verify that the changes have been correctly persisted in the database. This list view (Figure 6.2) is the same screen described in detail in Section 6.2; it is included here as a verification step that the CRUD interface in this section actually modifies the underlying `GAME` table through the stored procedures.

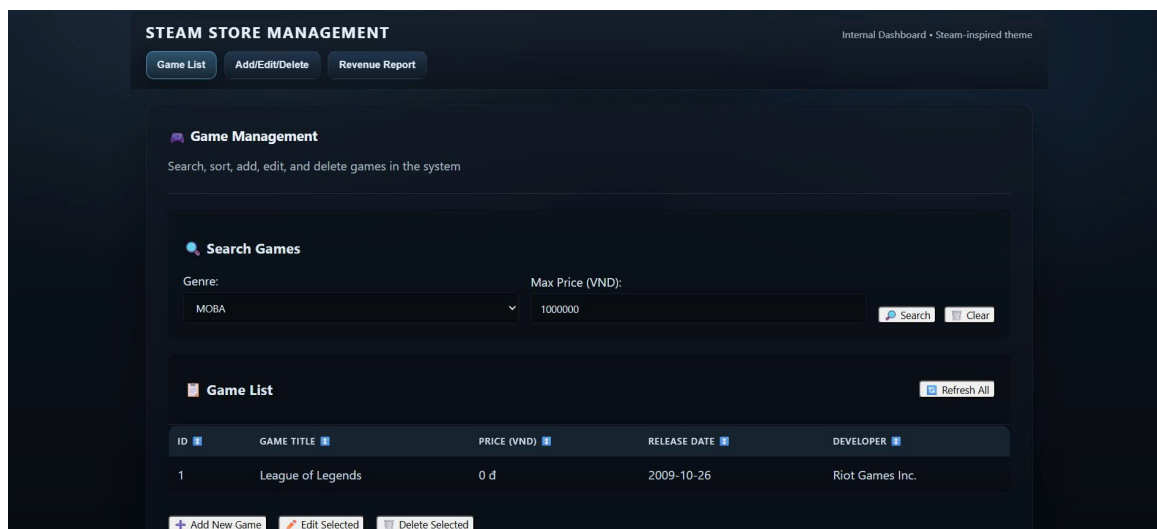


Figure 6.2: Game List view used to verify the result of CRUD operations performed in Figure 6.1

The CRUD interface demonstrates that users can manage database records through the application instead of executing SQL statements manually. It also shows the connection between



the frontend, backend API, and database stored procedures. By using stored procedures for data modification, the application maintains consistent validation rules between the database layer and the user interface.

6.2 Data List Interface

The data list interface presents the records retrieved from the `GAME` table together with related information from the `DEVELOPER` table. It is implemented in the React component `GameList.jsx` and is the central screen of the application: it integrates a data list, a search panel powered by the stored procedure `GetGamesByGenreAndPrice` from Section 5.3, and inline action buttons that reuse the CRUD stored procedures from Section 5.1. This consolidated design keeps the user workflow consistent because the same screen that displays the list also lets the user select a row and immediately add, update, or delete the record.

6.2.1 Interface Overview

When the page loads, three asynchronous calls are issued in parallel: `GET /api/games` to retrieve the full list of games, `GET /api/genres` to populate the genre dropdown, and `GET /api/developers` to populate the developer dropdown used in the modal form. The retrieved games are rendered as rows in an HTML table with the columns *ID*, *Game Title*, *Price (USD)*, *Release Date*, and *Developer*. A row becomes the *selected row* when the user clicks on it and is highlighted with a colored left border; the selected row is the target of the *Edit Selected* and *Delete Selected* actions.

Table 6.2: Main features of the data list interface

Feature	Description
List display	Loads all games via GET /api/games and renders them in a table with five columns.
Refresh	A <i>Refresh</i> button re-issues the games query to reflect any change made by another user.
Search by genre and max price	Calls the stored procedure GetGamesByGenreAndPrice via POST /api/search, returning only games that match both filters.
Clear search	Restores the full list by re-issuing GET /api/games.
Sortable columns	Each header column supports ascending/descending sort on the client side (see Section 6.2.3).
Add a new game	Opens a modal form bound to POST /api/games (sp_InsertGame).
Edit selected game	Opens the same modal in edit mode and submits via PUT /api/games/:id (sp_UpdateGame).
Delete selected game	Confirms the user's intent and issues DELETE /api/games/:id (sp_DeleteGame).
Inline messages	A red banner shows error messages and a green banner shows success messages; both can be dismissed manually or auto-hide after three seconds.

6.2.2 Search and Filtering

The search panel contains a *Genre* dropdown populated from the GENRE table and a *Max Price (USD)* numeric input. When the user clicks *Search*, the frontend validates that a genre is selected and that the price is a non-negative number. If validation fails, the request is not sent and an inline error message is displayed. Otherwise the frontend sends a JSON body { *genre*, *maxPrice* } to POST /api/search, and the backend forwards the parameters to the stored procedure GetGamesByGenreAndPrice. The backend returns the result rows, which replace the current list. If the procedure returns an empty result, the interface displays the friendly message “*No games found for genre X under Y USD*” so that the user can adjust the filter without confusion.



6.2.3 Sorting

In addition to the database-level filtering provided by the stored procedure, the interface supports client-side sorting on every column. When the user clicks a column header, the games array stored in the React state is sorted by the corresponding field. Successive clicks on the same header toggle between ascending and descending order. Sorting is applied locally so that re-sorting a list of fewer than a few hundred rows is instantaneous and does not require a round trip to the database.

6.2.4 Update and Delete Directly from the List

The two operations *update* and *delete* are deliberately performed against the row currently selected in the list. To update, the user clicks a row to mark it as the selected row, then clicks *Edit Selected*. The modal form opens pre-filled with the selected game's title, base price, and release date; when the user confirms, the frontend issues `PUT /api/games/:id`, which calls `sp_UpdateGame` on the database side. To delete, the user clicks a row, then clicks *Delete Selected*; a JavaScript `window.confirm` dialog asks for confirmation, and on agreement the frontend issues `DELETE /api/games/:id`, which calls `sp_DeleteGame`. Both operations invalidate the list and trigger an automatic re-fetch so that the table always reflects the latest state of the database.

6.2.5 Input Validation and Error Handling

The interface enforces validation at two levels. On the frontend, before any HTTP request is sent, it checks for empty titles, non-numeric or negative prices, missing release dates, and missing developer selection (for the insert form). On the backend, the stored procedures `sp_InsertGame`, `sp_UpdateGame`, and `sp_DeleteGame` perform the authoritative validation, including foreign key existence checks and the rule that a game already owned in a player's library cannot be deleted. Whenever a stored procedure raises a `SIGNAL SQLSTATE '45000'` error, the backend forwards the original message to the frontend, where it is displayed in the red error banner. This guarantees that the user always sees a clear, specific error message rather than a generic "request failed" notice.

Table 6.3: Examples of validation and error handling on the data list interface

User action	Validation layer	Resulting message
Click <i>Edit Selected</i> without selecting a row	Frontend	“Please select a game first.”
Submit the modal with an empty title	Frontend	“Game title cannot be empty.”
Submit the modal with negative price	Frontend	“Price must be a positive number.”
Insert a game whose developer_id does not exist	Backend (sp_InsertGame)	Original SQL error forwarded to the banner.
Delete a game that is already owned by a player	Backend (sp_DeleteGame)	“Cannot delete game that is already owned by players.”

6.2.6 Execution Result

The screenshots below illustrate the data list interface in operation. Figure 6.3 shows the default view after the page loads with the full list of games, the row currently selected by the user (highlighted in red), the action buttons *Add New Game*, *Edit Selected*, and *Delete Selected*, and the *Refresh* button. Figure 6.4 shows the result of calling the stored procedure *GetGamesByGenreAndPrice* with a filter that returns no matching record, including the friendly empty-state message produced by the frontend. Figure 6.5 shows the modal that opens when the user clicks *Edit Selected*. Figure 6.6 shows the browser confirmation dialog displayed before *sp_DeleteGame* is called. Figure 6.7 and Figure 6.8 demonstrate the two main error scenarios: clicking *Edit Selected* without selecting a row, and submitting the *Add New Game* modal with an empty title.

STEAM STORE MANAGEMENT Dashboard nội bộ • theme mô phỏng Steam

[Quản lý Game \(Hùng\)](#) [Thêm/Sửa/Xóa \(Ẩn\)](#) [Báo cáo Doanh thu \(Khôi\)](#)

Game List [Refresh](#)

ID	GAME TITLE	PRICE (USD)	RELEASE DATE	DEVELOPER
1	League of Legends	0.00	2009-10-26	Riot Games Inc.
2	Valorant	0.00	2020-06-01	Riot Games Inc.
3	Cyberpunk 2077	990000.00	2020-12-09	CD Projekt S.A.
4	Half-Life: Alyx	1200000.00	2020-03-22	Valve Corporation
5	Genshin Impact	0.00	2020-09-27	miHoYo Co., Ltd.
6	Grand Theft Auto V	500000.00	2015-04-13	Rockstar Games
7	Red Dead Redemption 2	1000000.00	2019-11-04	Rockstar Games
8	Final Fantasy VII Remake	1500000.00	2021-12-15	Square Enix Co., Ltd.

[+ Add New Game](#) [✎ Edit Selected](#) [🗑 Delete Selected](#)

Figure 6.3: Default view of the data list interface, with one game row selected

Search Games

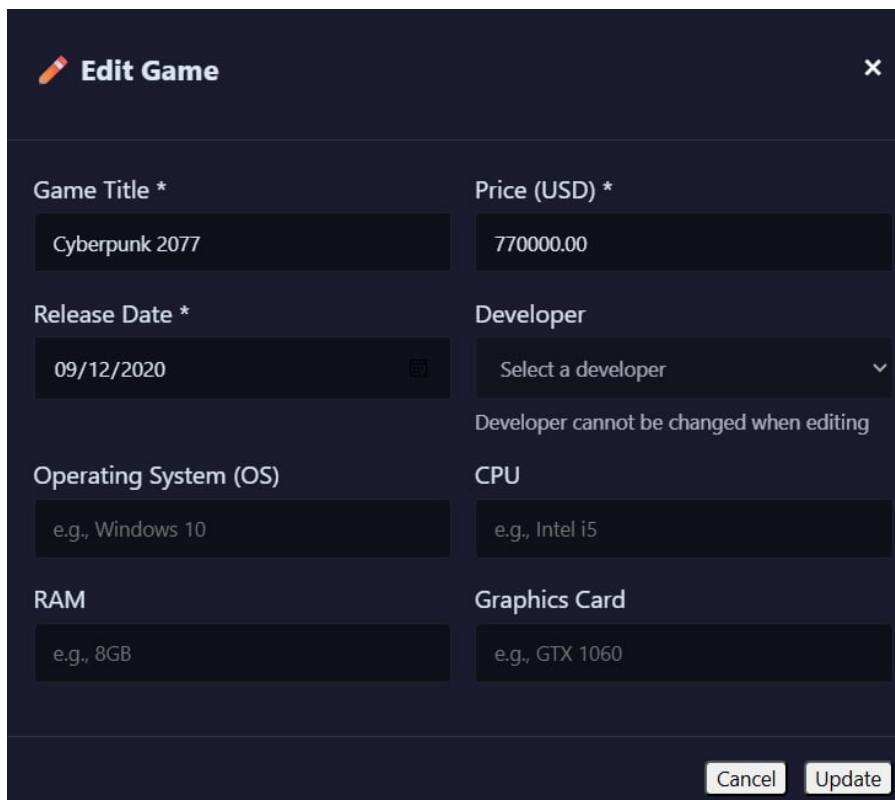
Genre: Max Price (USD): [Search](#) [Clear](#)

No games found for genre "Action" under 30 USD

Game List [Refresh](#)

ID	GAME TITLE	PRICE (USD)	RELEASE DATE	DEVELOPER
No data available				

Figure 6.4: Filtered list after invoking GetGamesByGenreAndPrice, showing the friendly empty-result message



Edit Game

Game Title * Cyberpunk 2077

Price (USD) * 770000.00

Release Date * 09/12/2020

Developer Select a developer

Developer cannot be changed when editing

Operating System (OS) e.g., Windows 10

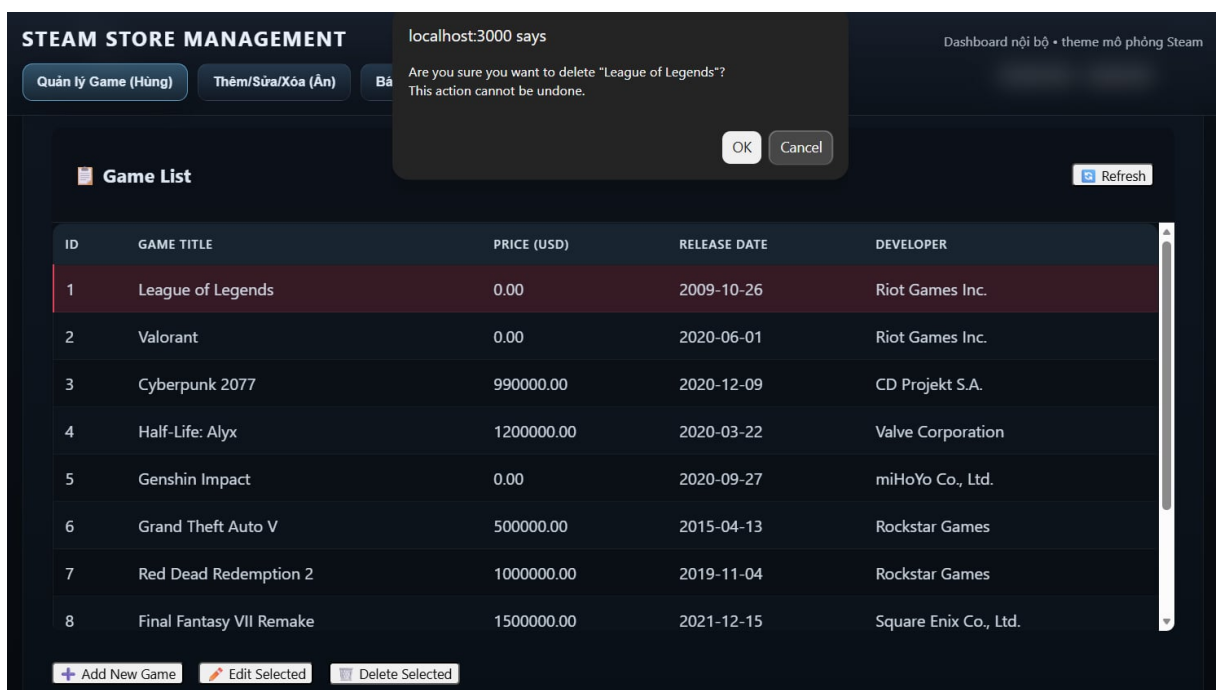
CPU e.g., Intel i5

RAM e.g., 8GB

Graphics Card e.g., GTX 1060

Cancel Update

Figure 6.5: Modal form opened by clicking *Edit Selected*, pre-filled with the selected game's data



STEAM STORE MANAGEMENT

localhost:3000 says

Dashboard nội bộ • theme mô phỏng Steam

Quản lý Game (Hàng) Thêm/Sửa/Xóa (Án) Bã

Are you sure you want to delete "League of Legends"?
This action cannot be undone.

OK Cancel

Game List

ID	GAME TITLE	PRICE (USD)	RELEASE DATE	DEVELOPER
1	League of Legends	0.00	2009-10-26	Riot Games Inc.
2	Valorant	0.00	2020-06-01	Riot Games Inc.
3	Cyberpunk 2077	990000.00	2020-12-09	CD Projekt S.A.
4	Half-Life: Alyx	1200000.00	2020-03-22	Valve Corporation
5	Genshin Impact	0.00	2020-09-27	miHoYo Co., Ltd.
6	Grand Theft Auto V	500000.00	2015-04-13	Rockstar Games
7	Red Dead Redemption 2	1000000.00	2019-11-04	Rockstar Games
8	Final Fantasy VII Remake	1500000.00	2021-12-15	Square Enix Co., Ltd.

Refresh

+ Add New Game Edit Selected Delete Selected

Figure 6.6: Browser confirmation dialog displayed before deleting the selected game

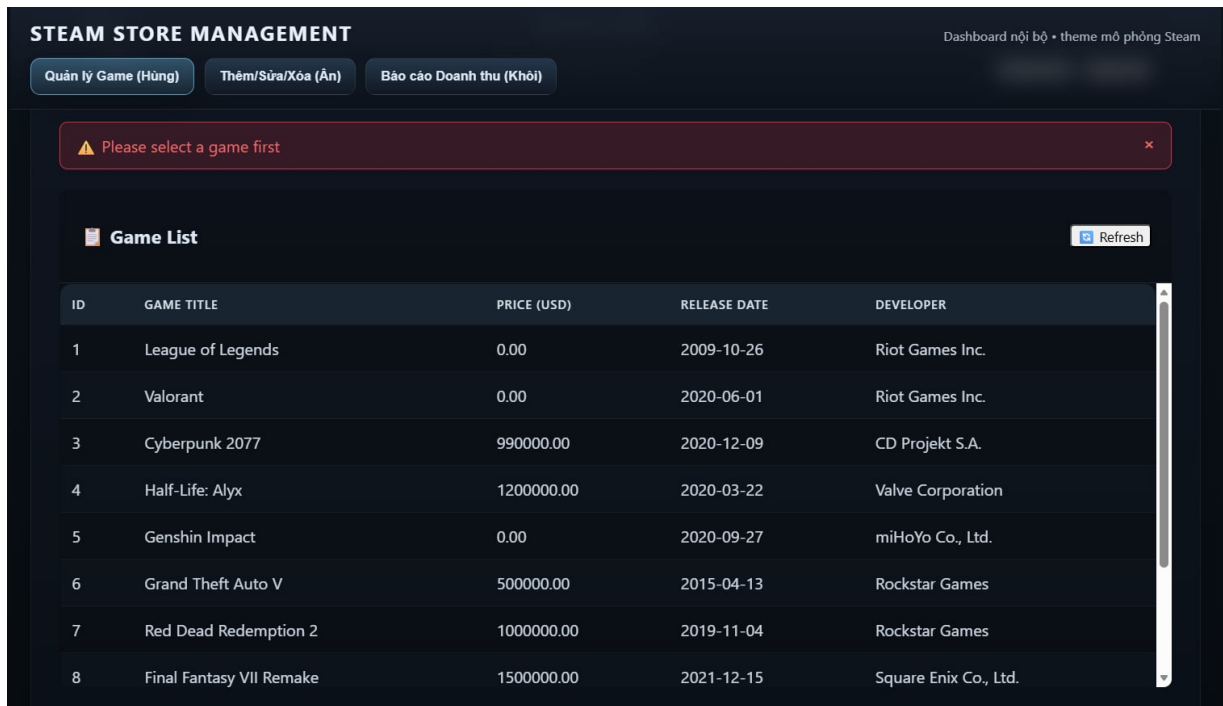


Figure 6.7: Frontend validation error when clicking *Edit Selected* with no row selected

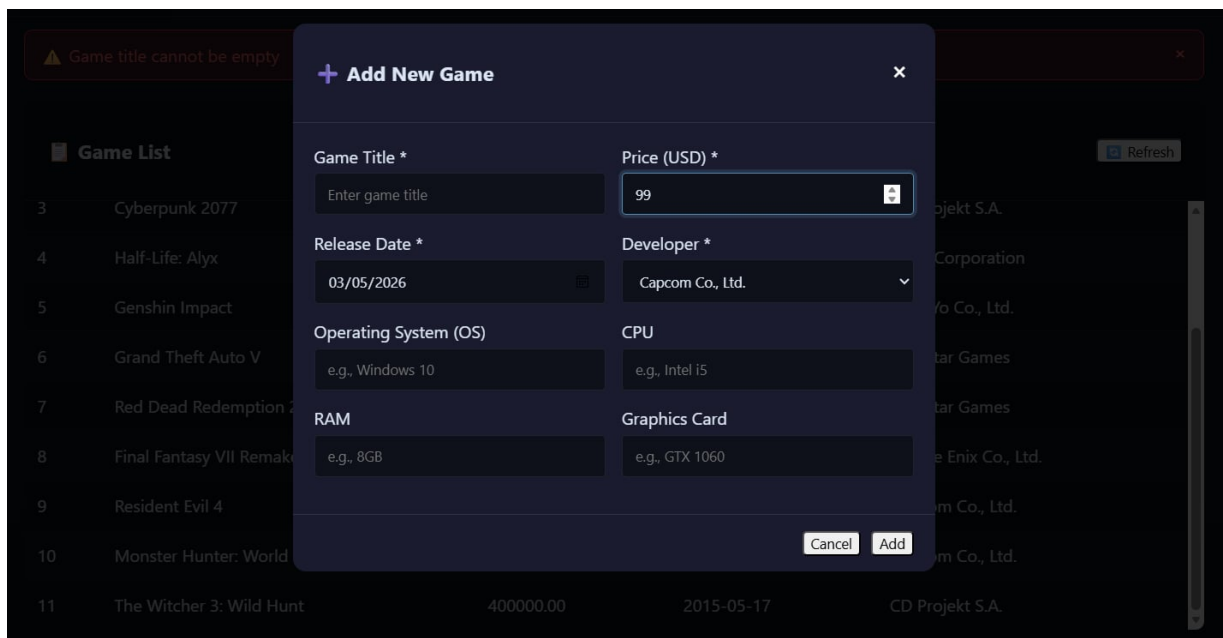


Figure 6.8: Frontend validation error when submitting the *Add New Game* modal with an empty title

6.3 Procedure and Function Demonstration Interface

This section presents the interface used to demonstrate database stored programs through the web application. In this implementation, the application provides a revenue report inter-



face that allows users to call a stored procedure from the frontend instead of executing SQL commands manually in MySQL Workbench.

The interface is implemented in the React component `RevenueReport.jsx`. It allows users to enter three input parameters: start date, end date, and minimum revenue. After the user submits the form, the frontend sends these parameters to the backend API. The backend then calls the stored procedure `GetDeveloperRevenue` and returns the result to the frontend for display.

6.3.1 Interface Purpose

The purpose of this interface is to demonstrate how stored procedures can be integrated into an application-level workflow. Instead of showing static database records only, this page allows users to generate a dynamic revenue report based on selected conditions.

The interface is designed for administrator or business analysis use cases. For example, an administrator can choose a date range and a minimum revenue threshold to identify developers whose total sales revenue exceeds that amount. This is a realistic feature for a Steam-like platform because the system needs to support revenue tracking, sales analysis, and developer performance monitoring.

6.3.2 Frontend Design

The frontend form contains three main input fields: `startDate`, `endDate`, and `minRevenue`. The default date range is set from 2023-01-01 to 2024-12-31, which matches the sample transaction data used in the database.

Before sending the request to the backend, the frontend performs a simple validation step. If the start date is later than the end date, an error message is displayed and the request is not sent. This prevents invalid input from reaching the backend and improves the user experience.

After the data is successfully retrieved, the interface displays the result in a table with three columns: developer name, total number of sales, and total revenue. The revenue value is formatted using the Vietnamese currency style to make the output easier to read.

6.3.3 Backend Processing

The backend is implemented using Node.js and Express. It provides an API endpoint at `/api/revenue-report`. This endpoint receives the input parameters from the frontend through query parameters and uses them to call the stored procedure `GetDeveloperRevenue`.

The backend does not manually calculate the revenue. Instead, the calculation logic is handled by the stored procedure in the database. This design keeps the business logic close to the data layer and allows the frontend to focus on input handling and result presentation.

Table 6.4: Workflow of the revenue report interface

Layer	Responsibility
Frontend	Provides input fields for date range and minimum revenue, validates input, and displays the report result.
Backend	Receives the request from the frontend and calls the stored procedure using the provided parameters.
Database	Executes <code>GetDeveloperRevenue</code> to calculate developer revenue from transaction and purchase data.

6.3.4 Stored Procedure Used

The interface calls the stored procedure `GetDeveloperRevenue`, which was implemented in the previous section. This procedure retrieves developers whose total revenue is greater than or equal to the selected minimum revenue within the selected date range.

The procedure uses joined data from the `DEVELOPER`, `GAME`, `CONTAINS`, and `TRANSACTION` tables. It then groups the result by developer and calculates both the total number of sold games and the total revenue.

Table 6.5: Stored procedure demonstrated in the revenue report interface

Stored Procedure	Purpose
<code>GetDeveloperRevenue</code>	Retrieves developers whose revenue exceeds a given threshold within a selected date range.

6.3.5 Execution Result

The revenue report interface was tested by running the frontend and backend servers locally. After opening the application in the browser, the user selects the date range and minimum revenue, then clicks the filter button to generate the report.

The result table displays the developers that satisfy the selected conditions. Each row contains the developer name, total number of sales, and total revenue. This confirms that the frontend, backend API, stored procedure, and MySQL database are connected successfully.

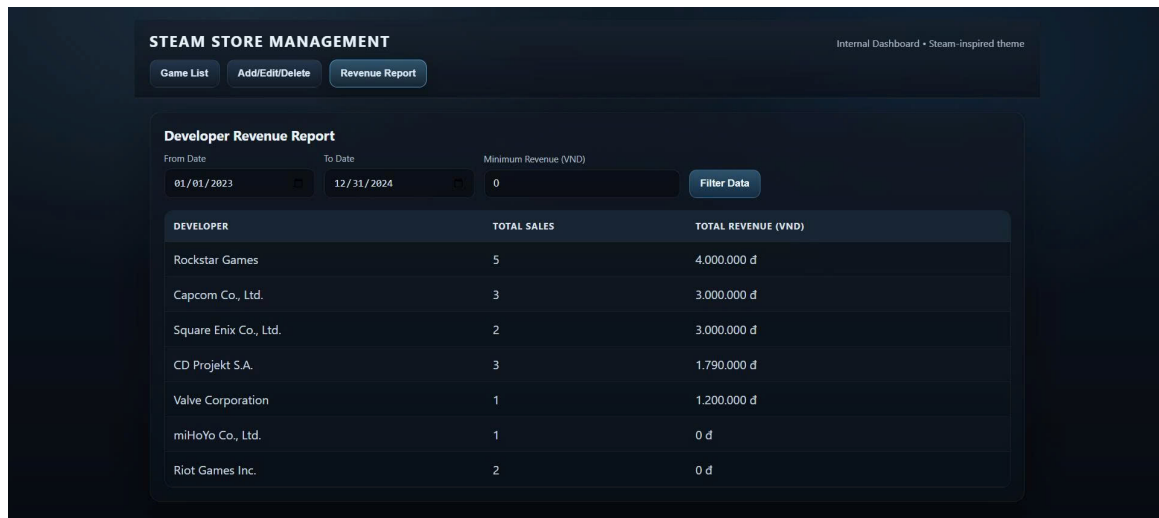


Figure 6.9: Revenue report interface for demonstrating the GetDeveloperRevenue stored procedure